

Guía de estudio: RLS, roles, middlewares y todo lo que los rodea

Qué es cada concepto, por qué se usó y dónde está en el código.
Escrita para quien parte de cero.

Contenido	Parte 1 · Conceptos desde cero Parte 2 · Profundización técnica
Rama	appcopio-multitenant
Estado del código	commit ae6e780 + cambios sin commitear
Generado	06-10-2026

Cómo usar esta guía

Este PDF reúne dos documentos del repositorio, en el orden en que conviene estudiarlos:

Parte	Archivo de origen	Para qué sirve
Parte 1	<code>docs/07_conceptos_multitenant_para_principiantes.md</code>	Los cimientos y cada concepto, uno por uno (qué es → por qué hace falta → dónde está), sin asumir ningún conocimiento previo. Empieza aquí.
Parte 2	<code>docs/00_guia_rls_desde_cero.md</code>	La misma historia con más profundidad: código SQL y TypeScript completo, los tropiezos reales (P1, P2, P19, P38/P39...) y la validación paso a paso.

Ruta de estudio sugerida

1. Lee la **Parte 1** completa, en orden. Las secciones A a C son lo esencial; si solo tienes tiempo para una, que sea la C.
2. Con `backend/src/auth/tenantContext.ts` y `backend/src/config/db.ts` abiertos, relea la sección C: son cortos y el código calza con el texto.
3. Pasa a la **Parte 2** cuando quieras ver el detalle y por qué fallaron las primeras versiones.

Qué verás repetido (a propósito)

Ambas partes explican RLS, `appcopio_app`, `withTenant` y `SECURITY DEFINER`. La Parte 1 lo hace desde cero y con analogías; la Parte 2, con el código real y los errores que lo moldearon. Leerlo dos veces desde ángulos distintos ayuda a fijarlo.

Qué no está en este PDF

El catálogo completo de los 40 problemas numerados está en `docs/02_implementacion_tecnica.md`, y los guiones de validación en `docs/03_guion_de_validacion.md` y `scripts/validar_multitenant.sh`. Son material de consulta, no de estudio lineal. Las reglas duras del proyecto están en `CLAUDE.md`.

Vigencia: el contenido describe el código en el estado indicado en la portada. Si el esquema o los middlewares cambian, edita los `.md` y vuelve a generar el PDF con `python scripts/generar_guia_pdf.py`.

Índice

Parte 1 · Conceptos desde cero

0. El mapa completo en 60 segundos	6
PARTE A — Los cimientos (lo que hay que saber antes de RLS)	6
A1. Multi-tenant y municipality_id	6
A2. Roles de Postgres (¡no son los roles de la app!)	7
A3. Transacciones (BEGIN / COMMIT / ROLLBACK)	8
A4. Pool de conexiones	9
A5. Variables de sesión y set_config (el "post-it" en la conexión)	9
A6. Funciones SQL, STABLE y triggers	10
PARTE B — Row-Level Security (RLS)	10
B1. Qué es RLS	10
B2. Los tres pasos, siempre en este orden	11
B3. ENABLE: qué hace y qué pasa si no hay políticas	11
B4. FORCE: qué hace y por qué existe	11
B5. Políticas: USING vs WITH CHECK (leer vs escribir)	12
B6. Políticas "por comando" (FOR SELECT, FOR INSERT...)	12
B7. Varias políticas se suman con OR (y por qué eso dio un bug real)	13
PARTE C — El backend: middlewares y cómo le cuentan a Postgres quién eres	13
C1. Qué es un middleware	13
C2. JWT y requireAuth — ¿quién eres?	13
C3. withTenant — contárselo a Postgres (el middleware central)	14
C4. Los tres modos de contexto	14
C5. El truco del pool interceptado (AsyncLocalStorage)	14
C6. Guardias de rol: otra capa, otra pregunta	15
PARTE D — SECURITY DEFINER: la ventanilla controlada	16
D1. El problema: RLS aísla demasiado bien	16
D2. Qué es	16
D3. Por qué es peligrosa, y cómo se hace segura	16
D4. Dónde se usa en AppCopio	16
PARTE E — Cómo se cubrieron las tablas: cuatro patrones	17
PARTE F — Colaboración intermunicipal (SuperEventos), en corto	18
PARTE G — Una petición de principio a fin, con ejemplo	18
PARTE H — Qué protege RLS y qué NO (límites honestos)	19
PARTE I — Trampas típicas al desarrollar (lo que más sorprende)	19
PARTE J — Cómo comprobar que de verdad aísla	20
Glosario rápido	20
Mapa de archivos	21
Orden de lectura recomendado	22

Parte 2 · Profundización técnica

1. Qué problema hay que resolver	23
Lo que pasa si solo agregas una columna	23

La pregunta central de toda la migración	24
2. Las tres estrategias, y por qué se eligió esta	24
La idea en una frase	24
El aislamiento quedó en tres capas	24
3. Row-Level Security explicado desde cero	25
Los tres pasos, siempre en este orden	25
ENABLE vs FORCE: la diferencia que arruina todo si se omite	26
USING vs WITH CHECK: leer vs escribir	26
Un UPDATE pasa por las dos	26
Varias políticas sobre la misma tabla se suman con OR	26
El truco de "RLS activo y ninguna política" = denegar todo	27
4. Por qué hubo que crear appcopio_app	27
La regla de PostgreSQL que lo obliga	27
El rol nuevo	28
Entonces, ¿quién usa cada rol?	28
Cómo comprobarlo en 5 segundos	29
5. Cómo la base de datos sabe de qué comuna eres	29
Variables de sesión: el recado que se le deja a la conexión	29
Las tres funciones de contexto	29
Por qué LOCAL — y por qué obliga a usar transacciones	30
La trampa de sintaxis que costó tiempo	30
6. Los middlewares: quién le cuenta a la base quién eres	31
requireAuth — ¿quién eres?	31
withTenant — contárselo a PostgreSQL	31
Los tres modos de contexto	32
Los guardias de rol, que son otra cosa	33
7. El interceptor del pool: cómo no reescribir 31 servicios	33
La solución: AsyncLocalStorage + un pool que miente amablemente	34
Qué se ganó con esto	35
El detalle del rollbackRequested	35
8. SECURITY DEFINER: la válvula de escape controlada	35
Qué significa	35
Caso 1 — El login, que no puede tener tenant todavía	36
Caso 2 — Encapsular una tabla cerrada a cal y canto	36
Caso 3 — Un trigger que necesita escribir donde el usuario no puede	36
Caso 4 — La colaboración entre comunas	37
Los permisos de las funciones	38
9. Cómo se cubrieron 38 tablas: cuatro patrones	39
Patrón A — Columna directa (lo más simple)	39
Patrón B — Catálogo dual: NULL significa "de todos"	39
Patrón C — Subconsulta al padre (tablas satélite)	40
Patrón D — Columna + trigger de herencia (tablas hoja de volumen)	40
Resumen de dónde está cada cosa	41
10. Los tropiezos reales (y qué enseña cada uno)	41
P1 — Activar RLS sobre Users rompió el login de todos	41
P2 — Fuga de tenant por montar withPublicContext a nivel de prefijo	41
P11 — Una comuna no podía leer sus propias prioridades	42
P12 — El conteo de participantes siempre daba 1	42
P19 — Fuga de conexiones que agotaba el pool y caía el proceso	42

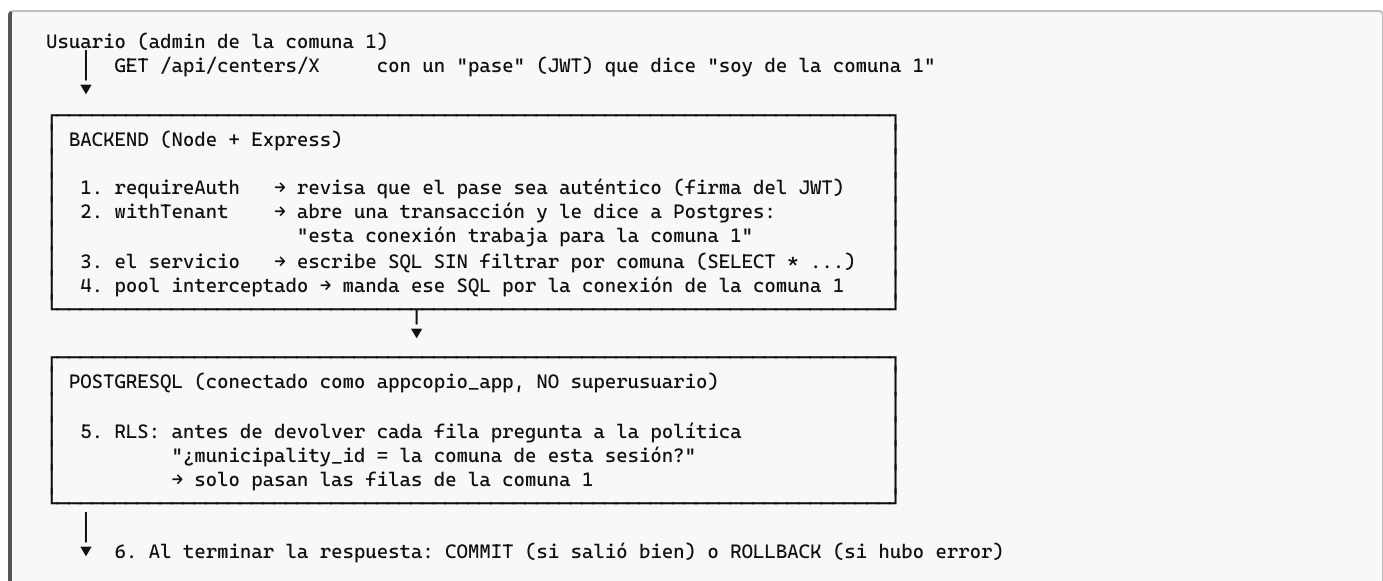
P38 y P39 — Las dos trampas de RLS al invitar comunas	42
El patrón común de todos estos errores	43
11. Cómo comprobar que el aislamiento es real	43
Paso 0 — Recrear el entorno desde cero	44
Paso 1 — El rol no debe poder saltarse RLS	44
Paso 2 — Todas las tablas con RLS activo y forzado	44
Paso 3 — La prueba más directa: sin tenant, sin datos	44
Paso 4 — Con tenant, solo lo propio	44
Paso 5 — Que las escrituras cruzadas exploten	45
Paso 6 — Por la API, que es lo que ve el usuario	45
El script que hace todo esto solo	45
12. Chuleta	45
Glosario	45
Errores de PostgreSQL que verás	46
El recorrido completo de una petición	46
Las diez reglas de oro	46
Dónde seguir leyendo	47

AppCopio multi-tenant: los conceptos, uno por uno (a prueba de tontos)

Esta guía asume que **no sabes nada** de RLS, roles de Postgres, transacciones ni middlewares. Cada concepto sigue el mismo orden: **qué es** → **por qué hace falta** → **dónde está en AppCopio**. Cuando ya lo tengas claro y quieras más detalle y los 40 problemas numerados, sigue con `00_guia_rijs_desde_cero.md` y `02_implementacion_tecnica.md`.

0. El mapa completo en 60 segundos

Antes de entrar al detalle, esta es la película completa. Cada pieza se explica después.



La idea en una frase: el backend no se encarga de filtrar por comuna; le *avisa* a Postgres de qué comuna es la petición, y **Postgres** filtra solo, siempre, en cada consulta.

PARTE A — Los cimientos (lo que hay que saber antes de RLS)

A1. Multi-tenant y `municipality_id`

Qué es. *Tenant* = "inquilino". Cada municipalidad es un tenant. *Multi-tenant* = un solo sistema, una sola base de datos, varios inquilinos que no deben verse entre sí. Como un edificio de departamentos: una estructura, muchas puertas con llave.

Cómo se distingue de quién es cada fila. Con una columna: `municipality_id`. Cada centro, usuario, familia, etc. lleva escrito a qué comuna pertenece. Esa columna se llama **discriminador**.

Por qué esta estrategia y no otra. Había tres caminos:

Opción	Cómo sería	Por qué no
Una base de datos por comuna	10 comunas = 10 bases	10 migraciones, 10 respaldos; y la colaboración entre comunas obligaría a consultar entre bases
Un esquema por comuna	1 base, 10 esquemas	Mismas migraciones multiplicadas
Una base, un esquema, columna <code>municipality_id</code>	Todo mezclado, cada fila etiquetada	Elegida: una sola migración, un solo respaldo, y entre comunas se puede compartir lo justo

El problema de la opción elegida. Todo está mezclado en las mismas tablas. Si `SELECT * FROM Centers` olvida el `WHERE municipality_id = 3`, devuelve centros de **todas** las comunas. Y AppCopio guarda nombres, RUT y composición familiar de damnificados: eso no es un bug visual, es una filtración de datos personales.

La pregunta que justifica toda la migración: **¿cómo hacemos que una consulta que olvida filtrar sea inofensiva en vez de peligrosa?** Todo lo que sigue es la respuesta.

A2. Roles de Postgres (¡no son los roles de la app!)

Ojo con la palabra "rol". En AppCopio hay dos cosas distintas con el mismo nombre:

	Rol de Postgres	Rol de la aplicación
Ejemplo	<code>postgres</code> , <code>appcopio_app</code>	Administrador, Trabajador Municipal, Super Administrador (<code>role_id</code> 4)
Quién lo tiene	La <i>conexión</i> a la base de datos	Cada <i>persona</i> que inicia sesión
Qué decide	Qué puede hacer el programa contra la base	Qué botones/acciones tiene permitida esa persona

En esta guía, "rol" a secas = rol de Postgres, salvo que diga "rol de la app".

Qué es un rol de Postgres. Una identidad con la que te conectas a la base. Cada rol tiene permisos (`GRANT SELECT ON tabla TO rol`) y atributos especiales. Dos atributos importan muchísimo aquí:

- `SUPERUSER`: puede hacer absolutamente todo y **se salta RLS siempre**.
- `BYPASSRLS`: no es superusuario, pero también se salta RLS.

El dueño de una tabla (owner) es el rol que la creó. Por defecto, el dueño también se salta RLS (a menos que uses `FORCE`, ver parte B).

Por qué importa en AppCopio. El proyecto original se conectaba como `postgres` (el superusuario que Docker crea por defecto). Si hubiéramos dejado eso, habríamos escrito todas las políticas de seguridad... y ninguna habría hecho nada, porque el superusuario las ignora. Peor: parecería que todo está protegido.

Dónde está. `db/002a_multitenant_schema.sql`, sección 12:

```
CREATE ROLE appcopio_app LOGIN PASSWORD '...';           -- sin SUPERUSER ni BYPASSRLS
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO appcopio_app;
```

Y en `backend/.env`: `DB_USER=appcopio_app`.

Quién usa cuál:

Rol	Lo usa	¿Pasa por RLS?	Para qué
<code>postgres</code> (superusuario)	Solo los scripts de <code>db/</code> al crear el contenedor	No	Crear tablas, roles y políticas; sembrar datos
<code>appcopio_app</code>	El backend, siempre	Sí	Toda la operación normal

Por qué son dos y no uno: crear un rol requiere ser superusuario. El superusuario arma el escenario y se va; el backend usa el rol limitado.

Cómo verificar (5 segundos):

```
SELECT rolname, rolsuper, rolbypassrls FROM pg_roles WHERE rolname = 'appcopio_app';
-- esperado: appcopio_app | f | f      (si ves 't' en alguna, el aislamiento es falso)
```

A3. Transacciones (`BEGIN` / `COMMIT` / `ROLLBACK`)

Qué es. Una transacción agrupa varias sentencias SQL como una sola operación "todo o nada":

```
BEGIN;                -- empieza el paquete
INSERT INTO ...;      -- paso 1
UPDATE ...;           -- paso 2
COMMIT;               -- confirma: ambos pasos quedan guardados
-- o bien:
ROLLBACK;             -- deshace: es como si nunca hubieran pasado
```

Por qué importa aquí. Dos razones, una buena y una obligatoria:

1. (Buena) Atomicidad: si crear un SuperEvento e invitar comunas falla a medias, no queda un SuperEvento huérfano.
2. (Obligatoria) Las variables de contexto del tenant (ver A5) **solo existen dentro de una transacción**. Sin transacción, el tenant se pierde.

Dónde está. `backend/src/auth/tenantContext.ts`: el middleware `withTenant` hace `BEGIN` al inicio del request y `COMMIT` / `ROLLBACK` al final (función `finishTx`).

A4. Pool de conexiones

Qué es. Abrir una conexión a Postgres es lento, así que el backend abre un puñado (aquí hasta 30) y las **reutiliza**: una petición pide una prestada, la usa y la devuelve; la siguiente petición recibe la misma (o cualquier otra) del montón.

```
pool = [ conexión1, conexión2, conexión3, ... ]
petición de Viña    → toma conexión2 → la devuelve
petición de Valpo   → toma conexión2 (¡la misma!) → ...
```

Por qué importa. Si pusiéramos "esta conexión es de Viña" de forma permanente, la siguiente petición (de Valparaíso) heredaría esa marca y vería datos de Viña. Por eso el tenant se fija con alcance **de transacción** (A5): se borra solo al terminar.

Dónde está. `backend/src/config/db.ts`, el `new Pool({...})`. Notarás `max: 30`, `statement_timeout` e `idle_in_transaction_session_timeout`: como ahora cada request retiene una conexión durante toda su duración (por la transacción), el límite por defecto de 10 se agotaría rápido (por ejemplo, subiendo un CSV con cientos de filas). Los timeouts evitan que una transacción olvidada bloquee una conexión para siempre.

A5. Variables de sesión y `set_config` (el "post-it" en la conexión)

El problema. El backend siempre se conecta con el *mismo* usuario de Postgres (`appcopio_app`), sea quien sea la persona que hizo la petición. Entonces Postgres no puede deducir "¿de qué comuna es esta petición?" a partir del usuario de la base. Hay que **decírselo**, en cada petición.

La herramienta. Postgres permite guardar variables propias (post-its) en la sesión:

```
SELECT set_config('app.current_tenant', '3', true); -- dejo el post-it: "comuna 3"
SELECT current_setting('app.current_tenant', true); -- lo leo → '3'
```

- El prefijo `app.` es solo convención para variables de la aplicación.
- El tercer parámetro `true` significa **LOCAL**: el post-it se borra solo en el `COMMIT` / `ROLLBACK`. Es una decisión de seguridad (ver A4) y es la razón por la que necesitamos transacciones (A3).
- En `current_setting`, el `true` significa "si no existe, devuelve NULL en vez de reventar".

Por qué `set_config(...)` y no `SET LOCAL app.current_tenant = $1`. El comando `SET` no acepta parámetros (`$1`), y pegar el valor a mano en el texto abriría la puerta a inyección SQL justo en el mecanismo que nos protege. `set_config` sí acepta parámetros. Está comentado en `tenantContext.ts`, función `setLocal`.

Las tres variables que usa AppCopio:

Variable	Significa	La fija
<code>app.current_tenant</code>	"soy de la comuna N"	<code>withTenant</code>

Variable	Significa	La fija
<code>app.is_superuser</code>	"soy el Super Administrador (administro la plataforma: no veo datos operativos de ninguna comuna)"	<code>withTenant</code>
<code>app.public_access</code>	"soy una visita anónima del mapa público"	<code>withPublicContext</code>

A6. Funciones SQL, `STABLE` y triggers

Función SQL. Un trozo de código guardado en la base, con nombre, que puedes llamar desde cualquier consulta. Para no repetir `current_setting(...)` en cada política, se envolvió en tres funciones pequeñas (`db/002a`, sección 6):

```
CREATE FUNCTION current_tenant() RETURNS INT AS $$
  SELECT NULLIF(current_setting('app.current_tenant', true), '')::INT;
$$ LANGUAGE sql STABLE;
-- is_superuser()      → ¿app.is_superuser = 'true'?
-- is_public_context() → ¿app.public_access = 'true'?
```

- `NULLIF(x, '')` convierte "vacío" en NULL, porque `''::INT` daría error.
- `STABLE` le promete al planificador "dentro de una consulta esto devuelve siempre lo mismo", así que la evalúa una vez en vez de una por fila (si no, una tabla de 100.000 filas haría 100.000 llamadas).

Trigger. Una función que Postgres ejecuta **automáticamente** cuando ocurre algo (por ejemplo, "antes de insertar una fila"). AppCopio usa triggers para:

- Generar IDs de centro tipo `VALPO-C001` (con contador por comuna).
- **Rellenar** `municipality_id` **automáticamente** en tablas hijas (ver parte E, patrón D).

PARTE B — Row-Level Security (RLS)

B1. Qué es RLS

Hasta RLS, los permisos eran **por tabla**: `GRANT SELECT ON Centers` = puedes leer la tabla entera o nada. **RLS baja un nivel: decide qué filas ves de esa tabla.**

Metáfora: `GRANT` te da (o no) la llave de la sala de archivadores. RLS decide qué carpetas de adentro puedes sacar.

El efecto práctico:

```
-- Lo que escribe el backend (sin filtro):
SELECT * FROM Centers;

-- Lo que ejecuta Postgres en realidad, porque hay una política activa:
SELECT * FROM Centers WHERE municipality_id = <la comuna de esta sesión>;
```

El programador no puede olvidar un filtro que no escribe.

B2. Los tres pasos, siempre en este orden

```
-- 1. ENCENDER RLS en la tabla.  
ALTER TABLE Centers ENABLE ROW LEVEL SECURITY;  
  
-- 2. FORZARLO también para el dueño.  
ALTER TABLE Centers FORCE ROW LEVEL SECURITY;  
  
-- 3. ABRIR la puerta justa con una política.  
CREATE POLICY centers_tenant_isolation ON Centers  
  USING      (municipality_id = current_tenant())  
  WITH CHECK (municipality_id = current_tenant());
```

(El Super Administrador no aparece: su contexto no fija comuna, así que sobre `Centers` no ve ninguna fila. Solo las tablas que su rol administra — `Users`, `Emergencies`, `SuperEvents`, sus participantes, `Municipalities` y `municipal_zones` — lo admiten.)

Esas líneas, repetidas sobre las tablas con datos de comuna, **son** la migración multi-tenant. Los archivos `db/002a` a `db/002d` son, en gran parte, eso.

B3. `ENABLE`: qué hace y qué pasa si no hay políticas

`ENABLE ROW LEVEL SECURITY` activa el filtrado. Lo crucial: **RLS es "denegar por defecto"**. Con RLS activo y *ninguna* política, nadie ve nada (la tabla parece vacía y no se puede escribir).

Eso es una herramienta, no un bug: para cerrar una tabla por completo, se activa RLS y **no se escribe ninguna política**. AppCopio lo hace con `RefreshTokens` (los tokens de sesión): ningún `SELECT` de la aplicación puede tocarla, y el único acceso son funciones controladas (ver parte D).

B4. `FORCE`: qué hace y por qué existe

Por defecto, `ENABLE` **no** aplica las políticas al **dueño** de la tabla (se asume que quien la creó debe poder verla entera). `FORCE ROW LEVEL SECURITY` quita esa excepción: el dueño también queda sujeto.

Para qué lo usamos, con honestidad. Hoy el backend se conecta como `appcopio_app`, que *no* es el dueño de las tablas (las creó `postgres`). Así que, hoy, `ENABLE` solo ya bastaría para el backend. `FORCE` es **cinturón y tirantes**: si algún día alguien conecta con el rol dueño, o cambia quién es el dueño, las políticas siguen aplicándose. Por eso es una regla dura del proyecto (CLAUDE.md, regla 3) y no hay ninguna tabla con `ENABLE` sin `FORCE`.

Resumen de quién se salta RLS:

Rol	¿Se salta RLS?
Superusuario (postgres)	Siempre , incluso con <code>FORCE</code>
Rol con <code>BYPASSRLS</code>	Siempre
Dueño de la tabla	Sí con <code>ENABLE</code> ; no con <code>ENABLE</code> + <code>FORCE</code>
Cualquier otro rol (appcopio_app)	Nunca

B5. Políticas: `USING` vs `WITH CHECK` (leer vs escribir)

Una política es una regla: "una fila es visible/modificable si se cumple esta condición". Tiene dos cláusulas, y esta es la distinción que más confunde:

Cláusula	Se aplica a	Mira	Si no se cumple
<code>USING</code>	<code>SELECT</code> , <code>UPDATE</code> , <code>DELETE</code>	La fila que ya existe	La fila simplemente no aparece (sin error)
<code>WITH CHECK</code>	<code>INSERT</code> , <code>UPDATE</code>	La fila como quedaría	Error (42501), la escritura se rechaza

Ejemplo de la asimetría:

```
-- Leo un centro de otra comuna:  
SELECT * FROM Centers WHERE center_id = 'OTRA-C001';  
--> 0 filas. Sin error: para ti ese centro no existe.  
  
-- Creo un centro a nombre de otra comuna:  
INSERT INTO Centers (name, municipality_id) VALUES ('Trampa', 99);  
--> ERROR: new row violates row-level security policy
```

Un `UPDATE` pasa por las dos: `USING` valida que la fila original sea tuya, `WITH CHECK` valida que no la dejes perteneciendo a otra comuna. Eso bloquea el truco de "regalar" una fila a otra comuna con `UPDATE ... SET municipality_id = 7`.

Por eso el backend traduce: error `42501` → HTTP 403; 0 filas → 404.

B6. Políticas "por comando" (`FOR SELECT` , `FOR INSERT ...`)

Si no dices `FOR ...` , la política vale para todos los comandos. Pero puedes acotarla:

```
CREATE POLICY centers_public_read ON Centers  
FOR SELECT -- SOLO lectura  
USING (is_public_context() AND is_active = TRUE);
```

Esto es lo que implementa la **regla 7 del proyecto**: las políticas que *amplían* la lectura (mapa público, colaboración entre comunas) son siempre `FOR SELECT` . Escribir nunca se amplía: `INSERT/UPDATE/DELETE` siguen exigiendo `municipality_id = current_tenant()` .

B7. Varias políticas se suman con OR (y por qué eso dio un bug real)

Las políticas son **permisivas**: si hay varias para el mismo comando, basta que **una** se cumpla. Se suman con OR.

`CenterItemPriority` tiene tres políticas de lectura: la propia, la pública y la intercomunal. Eso permite que cada contexto vea lo suyo.

El peligro, y por qué existe `withTenantOrPublic`. Si por error el contexto *público* (`app.public_access = true`) estuviera activo **dentro de una petición autenticada**, la política pública se sumaría (OR) a la del tenant y un admin vería los centros activos de **todas** las comunas. Eso pasó de verdad (problema P2): se había montado el middleware público sobre el prefijo entero `/api/centers`.

Por eso: **los contextos son mutuamente excluyentes** (tenant o público, jamás ambos), y los routers que mezclan endpoints públicos y privados aplican el middleware **ruta por ruta** (mira `backend/src/routes/centerRoutes.ts`, líneas ~793-815) y no por prefijo.

PARTE C — El backend: middlewares y cómo le cuentan a Postgres quién eres

C1. Qué es un middleware

Una función de Express que corre **antes** del manejador de la ruta y puede enriquecer la petición, cortarla (respondiendo un error) o dejarla pasar con `next()`. Se encadenan en orden, como una línea de control en un aeropuerto: documento → detector de metales → embarque.

```
// backend/src/index.ts
app.use('/api/emergencias', requireAuth, withTenant, emergencyRoutes);
//                               ①           ②           ③ las rutas
```

El **orden importa**: `withTenant` necesita que `requireAuth` ya haya llenado `req.user`.

C2. JWT y `requireAuth` — ¿quién eres?

JWT = un "pase" firmado que el servidor entrega al hacer login. Contiene datos del usuario (`user_id`, `role_id`, `municipality_id` ...) y una **firma** hecha con un secreto que solo el servidor conoce. Si alguien cambia un solo dato del pase (por ejemplo, su comuna), la firma deja de calzar y el pase se rechaza.

`requireAuth` (`backend/src/auth/middleware.ts`): lee `Authorization: Bearer <token>`, verifica la firma y deja el contenido en `req.user`. Sin token o vencido → 401 y la petición muere ahí.

Por qué importa para multi-tenant: el JWT es la **única fuente confiable** de la comuna. De ahí la **regla 5**: *nunca* se usa un `municipality_id` que venga del body de la petición (el cliente lo podría inventar). Siempre `req.user.municipality_id`.

El tipo del pase está en `backend/src/auth/tokens.ts` (`JwtUser`). Nota que `municipality_id` es `null` solo para el Super Administrador.

`optionalAuth`: igual, pero si no hay token no corta; sigue como anónimo. Para rutas de doble uso, como el listado de centros (mapa público y panel municipal).

C3. `withTenant` — contárselo a Postgres (el middleware central)

`backend/src/auth/tenantContext.ts`. Hace, en orden:

1. **Valida**: si el usuario no es superadmin y no tiene comuna → 401 `TENANT_MISSING`. (Antes caía a un tenant falso `-1` y el usuario veía todo vacío sin entender por qué.)
2. **Toma una conexión del pool** y hace `BEGIN`.
3. **Deja el post-it**: `set_config('app.current_tenant', '<su comuna>', true)`, o `app.is_superadmin = true` si es superadmin.
4. **Ejecuta el resto del request** con esa conexión (ver C5).
5. **Al terminar la respuesta**, `finishTx` hace `COMMIT` o `ROLLBACK` y devuelve la conexión al pool.

La regla de cierre: `ROLLBACK` si el status HTTP es `>= 400` (o si un servicio pidió rollback); `COMMIT` en el resto. Beneficio: una petición que falla no deja datos a medias.

C4. Los tres modos de contexto

Middleware	Cuándo se usa	Qué fija	Qué ve el request
<code>withTenant</code>	Rutas autenticadas (la gran mayoría)	<code>app.current_tenant</code> o <code>app.is_superadmin</code>	Lo de su comuna (el superadmin, solo lo que administra: usuarios, SuperEventos, emergencias para agrupar)
<code>withPublicContext</code>	Rutas públicas sin sesión	<code>app.public_access</code>	Solo lo que las políticas públicas permiten
<code>withTenantOrPublic</code>	Rutas de doble uso	Uno u otro , nunca ambos	Según haya sesión o no

Las rutas de **login** (`/api/auth`) no usan ninguno: aún no se sabe quién eres, así que tampoco la comuna. Cómo se resuelve eso: ver D1.

C5. El truco del pool interceptado (`AsyncLocalStorage`)

El problema práctico. `withTenant` fija el tenant en **una conexión concreta**. Pero el código de los servicios (31 archivos) está lleno de:

```
import pool from '../config/db';
const { rows } = await pool.query('SELECT * FROM Centers');
```

`pool.query()` normalmente toma **cualquier** conexión del pool, una que *no* tiene el post-it. Resultado: cero filas o comportamiento errático. Había que reescribir 31 servicios (8 de ellos importan `pool` directo, 23 reciben `db` por parámetro, y 13 archivos tienen sus propios `BEGIN/COMMIT`).

La solución, sin tocar ningún servicio. Dos piezas en `backend/src/config/db.ts`:

1. **AsyncLocalStorage**: una utilidad de Node que funciona como una "variable global *por petición*":
withTenant guarda ahí la conexión con el tenant fijado, y queda disponible en toda la cadena de llamadas de ese request sin pasarla por parámetro.
2. **Interceptar pool.query y pool.connect** (una sola vez, en ese archivo): si hay una conexión de request guardada, **redirigen todo a ella**.

```
(pool as any).query = (text, params, cb) => {
  const store = tenantStorage.getStore();
  if (store) {
    // Dentro de un request con tenant:
    // - BEGIN/COMMIT/ROLLBACK de servicios viejos → se ignoran (la transacción es del middleware)
    // - cualquier otra consulta → a la conexión con el tenant puesto
    ...
    return store.client.query(text, params, cb);
  }
  return originalQuery(text, params, cb); // fuera de un request: normal
};
```

Y **pool.connect()** devuelve un *Proxy* sobre la misma conexión, con **.release()** convertido en "no hacer nada" (lo libera el middleware al final; si el servicio la liberara a mitad, se perdería el tenant).

Detalle sutil: si un servicio hace su propio **ROLLBACK**, ese comando se ignora, pero se marca **rollbackRequested = true**. Sin eso, el middleware haría **COMMIT** al final y se guardaría justo lo que el servicio quiso descartar.

C6. Guardias de rol: otra capa, otra pregunta

No confundir:

	Pregunta	Quién lo resuelve
withTenant + RLS	"¿De qué comuna son los datos que puedes tocar?"	Postgres
requireRole , requireTenant , crearGuardaAdmin , requireSuperAdmin	"¿Tienes el cargo para esta acción?"	El backend

Un detalle que lo ilustra: dentro de **tu** comuna, RLS deja pasar a un Contacto Ciudadano igual que a un Administrador (ambos son de la misma comuna). Lo que impide que el Contacto Ciudadano cree centros es **requireCenterManagement**, no RLS. Está dicho en el comentario de **crearGuardaAdmin**: "*RLS aísla la comuna, no el rol dentro de ella.*"

requireTenant(req) (**backend/src/auth/requireUser.ts**) devuelve el **municipality_id** del JWT y es **la única fuente válida** para escrituras (regla 5). Lo usa, por ejemplo, **createCenter**.

Truco de orden: en **centerRoutes.ts** verás **requireAuth**, **requireCenterManagement**, **withTenant**, **createCenter**. Los guardias de rol van *antes* de **withTenant**, así una petición rechazada ni siquiera abre una transacción ni consume una conexión.

PARTE D — SECURITY DEFINER: la ventanilla controlada

D1. El problema: RLS aísla *demasiado bien*

A veces hay que leer algo legítimamente **fuera de tu tenant**, y desde RLS es imposible, porque hasta las subconsultas pasan por RLS. Ejemplos reales en AppCopio:

- **El login.** Para saber de qué comuna eres hay que leer `Users`. Pero `Users` tiene RLS por comuna. Pero aún no sabemos la comuna. Huevo y gallina: con RLS activo, el login fallaba para todo el mundo (problema P1).
- **La colaboración entre comunas:** mostrar en un tablero centros de *otras* comunas.

D2. Qué es

Toda función corre en uno de dos modos:

Modo	Corre con los permisos de...	¿Pasa por RLS?
<code>SECURITY INVOKER</code> (el normal)	quien la llama	Sí
<code>SECURITY DEFINER</code>	quien la creó (aquí, <code>postgres</code> , superusuario)	No

Metáfora: es una **ventanilla**. Tú no entras al archivo; le pides algo concreto a la persona que sí puede entrar, y ella decide qué entrega. **La función es la ventanilla, y su cuerpo es la política.**

D3. Por qué es peligrosa, y cómo se hace segura

Dentro de una función `SECURITY DEFINER` **no hay aislamiento**. Lo que la hace segura es que sea **estrecha**:

- Pocos parámetros y de tipo fijo (nada de texto SQL libre).
- Verificación explícita adentro (por ejemplo, `WHERE u.username = p_username`, exacto).
- Devuelve solo las columnas necesarias (el límite está en la *firma* de la función, no en el código TypeScript, así que nadie lo amplía por descuido).
- `SET search_path = public`, para que nadie anteponga un esquema falso y secuestre la función.
- `REVOKE ALL ... FROM PUBLIC; GRANT EXECUTE ... TO appcopio_app;`, porque por defecto *cualquiera* puede ejecutar una función nueva.

D4. Dónde se usa en AppCopio

Función	Archivo	Para qué
<code>auth_lookup_user(username)</code> / <code>auth_lookup_user_by_id</code>	002a	Login y <code>/auth/me</code> : leer <code>Users</code> sin tenant, acotado a un usuario exacto
<code>generate_center_id()</code> (trigger)	002a	Mover el contador <code>center_seq_counter</code> de <code>Municipalities</code> , tabla que solo el superadmin puede modificar
<code>public_center_occupancy</code>	002a	Ocupación pública de un centro, sin exponer personas

Función	Archivo	Para qué
<code>refresh_token_*</code> (7 funciones)	<code>002c</code>	Único acceso a <code>RefreshTokens</code> (tabla cerrada sin políticas)
<code>super_event_shared_centers</code> y similares	<code>002d</code>	Colaboración entre comunas: devuelve solo ubicación, capacidad, % de llenado y estado
<code>notify_super_event_invitation</code> , <code>notify_support_offer</code>	<code>002d</code>	Escribir un aviso en otra comuna, verificando antes que quien llama tenga derecho

Ejemplo de ventanilla segura (`002d`): `super_event_shared_centers` verifica que el evento siga abierto, que la comuna dueña del centro participe, **y que quien lee también participe**. Y su firma solo devuelve columnas permitidas por la regla 6: **nunca** `Persons`, `FamilyGroups`, `CentersDescription` ni cantidades de inventario.

Un detalle que conviene tener presente: como el creador de estas funciones es `postgres` (superusuario), `FORCE ROW LEVEL SECURITY` **no las frena**. No hay nada que las limite salvo su propio código. Por eso se escriben tan estrechas.

PARTE E — Cómo se cubrieron las tablas: cuatro patrones

No todas las tablas son iguales, así que se usó el patrón más barato en cada caso.

Patrón	Para qué tablas	Cómo funciona	Ejemplo
A. Columna directa	Tablas "raíz" con su propio <code>municipality_id</code>	Política <code>municipality_id = current_tenant()</code> (solo <code>Users</code> suma políticas aparte para el superadmin)	<code>Users</code> , <code>Centers</code> , <code>Persons</code> , <code>FamilyGroups</code> , <code>Datasets</code>
B. Catálogo dual	Productos, categorías, plantillas, zonas	<code>municipality_id IS NULL</code> = "global, de todos"; si no, solo de esa comuna	<code>Products</code> , <code>Categories</code>
C. Subconsulta al padre	Tablas satélite pequeñas	La política pregunta al padre: <code>dataset_id IN (SELECT dataset_id FROM Datasets)</code> . Esa subconsulta también pasa por RLS , así que hereda el aislamiento solo	<code>DatasetFields</code> , <code>TemplateFields</code> , <code>ResourceBoxItems</code>
D. Columna + trigger de herencia	Tablas "hoja" de mucho volumen	Llevar su propio <code>municipality_id</code> , rellenado automáticamente por un trigger que lo copia del centro padre	<code>InventoryLog</code> , <code>UpdateRequests</code> , <code>CenterAssignments</code>

Por qué cuatro y no uno solo:

- El patrón A es el más rápido y simple, pero exige una columna.
- El C evita agregar columnas, pero cada lectura hace una subconsulta: aceptable en tablas chicas, caro en las grandes.
- El D evita esa subconsulta en tablas grandes, a cambio de una columna extra.
- El B existe porque hay datos que legítimamente son compartidos (el catálogo de productos base) y a la vez permiten productos propios por comuna.

Una sutileza del patrón D (comentada en 002c): el trigger corre como *invocador* (no SECURITY DEFINER) a **propósito**. Si intentas registrar inventario sobre un centro ajeno, la subconsulta al padre pasa por RLS, no lo ve, devuelve NULL, y la restricción NOT NULL rechaza la fila. El aislamiento de escritura se aplica "gratis".

Persons es un caso especial: podría deducir su comuna por subconsulta, pero es dato sensible que **nunca cruza de comuna**, así que tiene columna propia y un WITH CHECK real.

PARTE F — Colaboración intermunicipal (SuperEventos), en corto

El aislamiento total no alcanza: un incendio puede cruzar límites comunales. La respuesta no es "romper RLS" sino **abrirlo con precisión**:

- 1. **Solo lectura se amplía; escritura nunca** (regla 7). Las políticas ampliadas son FOR SELECT. Para escribir en otra comuna (invitar, avisar) hay funciones SECURITY DEFINER que verifican antes.
- 2. **Solo se comparte lo mínimo** (regla 6): ubicación, capacidad, % de llenado, estado operacional, prioridades agregadas. Las personas **jamás**.
- 3. **Ambos lados deben participar**, y el SuperEvento debe estar abierto (los "cuatro candados" de super_event_shared_centers).
- 4. **El límite vive en la base, no en TypeScript**. Si alguien edita el backend, no puede exponer más columnas por accidente.

Consecuencia de RLS que sorprende: **los agregados cuentan solo lo visible**. Un COUNT(*) de participantes de un SuperEvento devolvía 1 (solo veías tu propia fila). Se resolvió con una ventanilla super_event_participants_of(...).

PARTE G — Una petición de principio a fin, con ejemplo

Escenario: Ana, administradora de la comuna 1, abre su listado de centros. GET /api/centers/status/active, con Authorization: Bearer eyJ...

Paso	Qué pasa	Dónde
1	Express recibe la petición y corre requireAuth: verifica la firma. El JWT dice municipality_id: 1, role_id: 1. Lo deja en req.user.	auth/middleware.ts
2	Corre withTenant: toma una conexión del pool, BEGIN, y set_config('app.current_tenant', '1', true). Guarda la conexión en AsyncLocalStorage.	auth/tenantContext.ts
3	Corre el handler/servicio, que ejecuta SELECT * FROM Centers WHERE is_active = true. Sin filtro de comuna.	routes/, services/
4	pool.query está interceptado: ve que hay conexión de request y manda la consulta a esa conexión.	config/db.ts

Paso	Qué pasa	Dónde
5	Postgres aplica la política <code>centers_tenant_isolation</code> : <code>municipality_id = current_tenant()</code> (= 1). Solo salen los centros de la comuna 1.	<code>db/002a</code> sección 7
6	El handler responde 200. <code>finishTx</code> hace <code>COMMIT</code> y devuelve la conexión al pool (con el post-it ya borrado).	<code>auth/tenantContext.ts</code>

¿Y si Ana (o un atacante con su token) intenta ver un centro de la comuna 2? `SELECT * FROM Centers WHERE center_id = 'COMUNA2-C001'` → **0 filas**; el servicio responde 404. Nunca se devuelven datos. Y si intenta **crear** algo a nombre de la comuna 2 en el body, el backend lo ignora (usa `requireTenant`), y aunque no lo ignorara, el `WITH CHECK` lo rechazaría con error 42501 → 403.

¿Y si el programador olvidó el `WHERE`? Da igual: el filtro lo pone Postgres.

PARTE H — Qué protege RLS y qué NO (límites honestos)

Para no quedarte con una falsa sensación de seguridad:

✅ RLS sí protege	❌ RLS no protege
Un <code>WHERE</code> olvidado en una consulta	Un backend comprometido o con una inyección SQL que ejecute <code>set_config('app.current_tenant', ...)</code> con otro valor: Postgres confía en lo que el backend le dice
Que un admin lea o escriba datos de otra comuna por un bug	Permisos de rol dentro de la misma comuna (eso son los guardias del backend)
Escrituras a nombre de otra comuna	Funciones <code>SECURITY DEFINER</code> mal escritas: dentro de ellas no hay aislamiento
Que un cliente falsifique su comuna en el body (el JWT firmado manda)	Conectar como superusuario o con <code>BYPASSRLS</code> : ahí todo es decorativo

Moraleja: por eso hay **tres capas** (JWT firmado → `withTenant` → RLS) más los guardias de rol, y por eso las reglas del proyecto insisten tanto en no usar superusuario y en mantener estrechas las funciones `SECURITY DEFINER`.

PARTE I — Trampas típicas al desarrollar (lo que más sorprende)

1. **"Mi consulta devuelve 0 filas."** Antes de culpar al `WHERE`, sospecha de la política: ¿estás dentro de `withTenant`? ¿la política permite leer eso? Con RLS, lo que no está permitido expresamente **no existe** (problema P11).
2. Un `pool.query()` **suelto fuera de un request** (scripts, arranque) no tiene tenant → cero filas. Es esperable, no un bug.

3. `INSERT ... ON CONFLICT` y `RETURNING` pueden fallar bajo RLS cuando la fila nueva no sería visible para ti (problemas P38/P39 en invitaciones de SuperEventos).
4. Un `COUNT/SUM` bajo RLS cuenta solo lo visible. Si necesitas el total real, hace falta una ventanilla.
5. No guardes estado por petición en un objeto que el pool reutilice (como el `PoolClient`). Se hizo una vez y causó fuga de conexiones (P19); el estado vive ahora en el store de `AsyncLocalStorage`.
6. No agregues `console.log` permanentes para depurar RLS (regla 9 del proyecto).
7. Cada cambio de esquema es un `.sql` nuevo en `db/`; se recrea todo con `docker compose down -v && docker compose up`. Sin `-v`, los scripts de `db/` no vuelven a correr (Docker solo los ejecuta con el volumen vacío) y estarías probando el esquema viejo.

PARTE J — Cómo comprobar que de verdad aísla

Lo peligroso de RLS es que una implementación rota **se ve igual que una correcta** mientras pruebes con un solo usuario. Pruebas rápidas, de la más importante a la más detallada:

```
# 1. El rol no se salta RLS (esperado: f | f)
docker compose exec -T db psql -U postgres -d appcopio_mt_db -tAc \
  "SELECT rolsuper, rolbypassrls FROM pg_roles WHERE rolname='appcopio_app'"

# 2. Sin tenant fijado, la app no ve NADA (esperado: 0)
docker compose exec -T db psql -U appcopio_app -d appcopio_mt_db -tAc \
  "SELECT COUNT(*) FROM Centers"

# 3. Con tenant fijado, solo lo de esa comuna (cambia el 1 por otro y el número cambia)
docker compose exec -T db psql -U appcopio_app -d appcopio_mt_db -tAq \
  -c "BEGIN" -c "SELECT set_config('app.current_tenant','1',true)" \
  -c "SELECT COUNT(*) FROM Centers" -c "COMMIT"
```

Y el script completo, que además prueba la API, los permisos, la colaboración y el canal público (necesita la base recién recreada):

```
docker compose down -v && docker compose up -d
bash scripts/validar_multitenant.sh
```

Glosario rápido

Término	Significa
Tenant	Cada organización que usa el sistema (aquí, una municipalidad)
Discriminador	La columna que dice de quién es cada fila: <code>municipality_id</code>
RLS	Row-Level Security: Postgres filtra <i>filas</i> según una regla, automáticamente
Política (<code>POLICY</code>)	La regla concreta de RLS. Varias sobre la misma tabla se suman con OR

Término	Significa
<code>ENABLE</code>	Enciende RLS (no aplica al dueño de la tabla)
<code>FORCE</code>	Lo aplica también al dueño
<code>USING</code>	Qué filas existentes puedes ver/tocar. Falla → la fila no aparece
<code>WITH CHECK</code>	Cómo puede quedar una fila que escribes. Falla → error
Superusuario / <code>BYPASSRLS</code>	Rol que se salta RLS siempre; el backend jamás debe usarlo
<code>appcopio_app</code>	Rol de Postgres sin privilegios especiales, el que usa el backend
Transacción	Grupo de sentencias "todo o nada" (<code>BEGIN</code> ... <code>COMMIT</code> / <code>ROLLBACK</code>)
Pool	Conjunto de conexiones reutilizables a la base
<code>set_config(..., true)</code>	Deja una variable de sesión que vive solo hasta el fin de la transacción
JWT	"Pase" firmado con los datos del usuario, incluida su comuna
Middleware	Función de Express que corre antes del manejador de la ruta
<code>AsyncLocalStorage</code>	Variable "global por petición" de Node
<code>SECURITY DEFINER</code>	Función que corre con permisos de su creador: no pasa por RLS ("ventanilla")
Trigger	Función que Postgres ejecuta sola ante un evento (ej. antes de insertar)

Mapa de archivos

Qué buscas	Dónde
Tablas originales (single-tenant)	<code>db/001_tablas.sql</code>
<code>Municipalities</code> , rol <code>appcopio_app</code> , funciones <code>current_tenant()</code> , RLS de las tablas núcleo	<code>db/002a_multitenant_schema.sql</code>
Notificaciones, lecturas de prioridades	<code>db/002b_emergencias_y_notificaciones.sql</code>
Tablas restantes, triggers de herencia, <code>RefreshTokens</code>	<code>db/002c_qls_tablas_restantes.sql</code>
SuperEventos y colaboración intermunicipal	<code>db/002d_supereventos.sql</code>
Pool interceptado + <code>AsyncLocalStorage</code>	<code>backend/src/config/db.ts</code>
<code>withTenant</code> , <code>withPublicContext</code> , <code>withTenantOrPublic</code> , <code>finishTx</code>	<code>backend/src/auth/tenantContext.ts</code>
<code>requireAuth</code> , <code>optionalAuth</code> , <code>requireRole</code>	<code>backend/src/auth/middleware.ts</code>
<code>requireTenant</code> , <code>crearGuardaAdmin</code>	<code>backend/src/auth/requireUser.ts</code>
Forma del JWT	<code>backend/src/auth/tokens.ts</code>
Qué rutas llevan qué middleware	<code>backend/src/index.ts</code> (líneas ~107-146) y <code>routes/centerRoutes.ts</code>

Qué buscas	Dónde
Las reglas duras del proyecto	<code>CLAUDE.md</code>

Orden de lectura recomendado

1. Esta guía, de arriba abajo (Partes A → C son lo esencial).
2. `docs/00_guia_qls_desde_cero.md` para más profundidad y ejemplos de SQL.
3. Abre `tenantContext.ts` y `db.ts` junto a la Parte C; son cortos y quedan claros.
4. `docs/02_implementacion_tecnica.md` cuando quieras ver los problemas reales y cómo se resolvieron.

Guía multi-tenant y RLS en AppCopio

Guía de entrada, a prueba de desinformados. Asume que conoces el AppCopio single-tenant, pero **nada** sobre Row-Level Security. Los otros documentos de esta carpeta suponen que ya leíste este.

AppCopio pasó de servir a una municipalidad a servir a muchas sobre la misma base de datos, y el aislamiento entre ellas no se dejó en manos del código de aplicación sino de PostgreSQL. Esta guía explica desde cero qué es Row-Level Security, por qué obligó a crear un usuario de base de datos nuevo, qué hace cada middleware y cómo encaja todo.

1. Qué problema hay que resolver

Multi-tenant significa: un solo sistema, una sola base de datos, un solo despliegue — pero varias organizaciones (aquí, varias municipalidades) usándolo a la vez sin verse entre sí. Cada organización se llama **tenant** (inquilino). El edificio es uno; los departamentos están cerrados con llave.

El AppCopio que ya conoces era **single-tenant**: toda la base de datos pertenecía implícitamente a una municipalidad. `SELECT * FROM Centers` devolvía "los centros", sin más, porque no había otros. Esa suposición está escrita en cientos de consultas SQL a mano por todo el backend.

Lo que pasa si solo agregas una columna

El instinto es agregar `municipality_id` a cada tabla y filtrar por ella. El problema es que eso convierte el aislamiento en **una cuestión de disciplina humana**: basta que una consulta de las cientos que hay olvide el `WHERE` para que una comuna vea los datos de otra.

```
-- Lo que se esperaba que escribiera siempre el programador
SELECT * FROM Centers WHERE municipality_id = 3;

-- Lo que escribió el martes a las 19:40, apurado
SELECT * FROM Centers;  -- ← fuga de datos entre municipalidades
```

Y no es una fuga cualquiera. AppCopio guarda nombres, RUT y composición familiar de personas damnificadas. Un `WHERE` olvidado no es un bug de presentación: es un incidente de datos personales.

Hay además una vuelta de tuerca propia del dominio: las comunas a veces **tienen que** colaborar (un incendio que cruza límites comunales). O sea que la respuesta no puede ser "aislar y listo": hace falta un aislamiento que sepa abrirse de forma controlada, para ciertos datos, entre ciertas comunas, durante cierto tiempo.

La pregunta central de toda la migración

¿Cómo se consigue que una consulta que **olvida** filtrar sea inofensiva, en vez de peligrosa?

Todo lo que viene a continuación —RLS, un usuario de base de datos nuevo, los middlewares, las funciones especiales— es la respuesta a esa única pregunta.

2. Las tres estrategias, y por qué se eligió esta

Para hacer multi-tenant una base relacional hay tres caminos clásicos.

Estrategia	Cómo se ve	Aislamiento	Costo	Por qué se descartó
Una base por comuna	10 comunas = 10 bases	Máximo	Alto: 10 migraciones, 10 respaldos	Inviabile para un equipo pequeño; la colaboración intercomunal exigiría consultas entre bases
Un esquema por comuna	1 base, 10 esquemas	Alto	Medio: 10 esquemas que sincronizar	Mismo problema de migraciones multiplicadas
Esquema compartido + columna discriminadora	1 base, 1 esquema, <code>municipality_id</code>	Depende del mecanismo	Bajo: una migración, un respaldo	Elegida

Se eligió la tercera, pero **con una condición**: que el aislamiento no dependa del código de aplicación. Y ahí entra Row-Level Security.

La idea en una frase

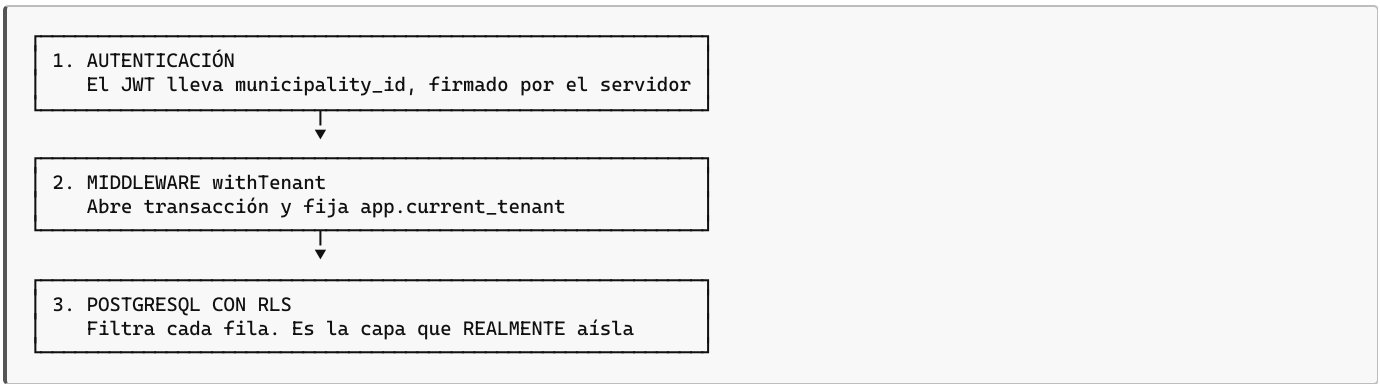
En vez de pedirle al programador que escriba el filtro, se le pide **al motor de PostgreSQL** que lo agregue solo, en cada consulta, siempre, sin excepción.

```
-- Lo que escribe el backend (sin filtro):  
SELECT * FROM Centers;  
  
-- Lo que EJECUTA PostgreSQL, porque hay una política RLS activa:  
SELECT * FROM Centers WHERE municipality_id = <la comuna de esta sesión>;
```

El programador no puede olvidarse de algo que no escribe. Eso convierte la consulta descuidada del ejemplo anterior en una consulta **inofensiva**: devuelve solo lo propio, igual que si hubiera filtrado bien.

El aislamiento quedó en tres capas

Ninguna capa se confía sola; cada una tapa lo que la anterior podría dejar pasar.



La capa 1 dice **quién eres** y no se puede falsificar (va firmada). La capa 2 se lo **cuenta a la base**. La capa 3 es la que **realmente** impide leer de más, y funciona aunque las dos primeras tuvieran un bug de filtrado.

3. Row-Level Security explicado desde cero

RLS = Row-Level Security = seguridad a nivel de fila. Es una función nativa de PostgreSQL desde la versión 9.5. Hasta ahora conocías permisos a nivel de *tabla* (`GRANT SELECT ON Centers TO alguien`). RLS baja un nivel: permite decir **qué filas** de esa tabla puede ver o tocar cada quien.

La metáfora: `GRANT` decide si tienes llave de la sala de archivadores. RLS decide qué carpetas de adentro puedes sacar.

Los tres pasos, siempre en este orden

```
-- PASO 1: encender RLS en la tabla.  
-- Efecto inmediato y brutal: sin políticas, la tabla queda VACÍA para todos.  
-- "Niega todo salvo lo que una política permita expresamente."  
ALTER TABLE Centers ENABLE ROW LEVEL SECURITY;  
  
-- PASO 2: forzarlo también para el DUEÑO de la tabla (explicado más abajo).  
ALTER TABLE Centers FORCE ROW LEVEL SECURITY;  
  
-- PASO 3: abrir la puerta justa con una política.  
CREATE POLICY centers_tenant_isolation ON Centers  
  USING      (municipality_id = current_tenant())  
  WITH CHECK (municipality_id = current_tenant());
```

El Super Administrador no aparece en la política: su contexto no fija comuna, así que sobre `Centers` no ve ninguna fila. Solo las tablas que su rol administra (`Users`, `Emergencies`, `SuperEvents` y sus participantes, `Municipalities` y `municipal_zones`) lo admiten, y cada una lo declara en su propia política.

Esas tres líneas, repetidas sobre 38 tablas, **son** la migración multi-tenant. Todo lo demás es consecuencia.

ENABLE vs FORCE : la diferencia que arruina todo si se omite

- `ENABLE ROW LEVEL SECURITY` activa las políticas... **para todos menos para el dueño de la tabla**. PostgreSQL asume que quien creó la tabla debe poder verla entera.
- `FORCE ROW LEVEL SECURITY` elimina esa excepción: **también el dueño queda sujeto a las políticas**.

Sin `FORCE`, si el backend se conectara con el usuario que creó las tablas, todas las políticas serían decorativas: las vería pasar de largo. Por eso en este repo **no hay una sola tabla con `ENABLE` sin `FORCE`**.

USING vs WITH CHECK : leer vs escribir

Esta es la distinción que más cuesta al principio, y es simple:

Cláusula	Cuándo actúa	Sobre qué fila	Si falla
<code>USING</code>	<code>SELECT</code> , <code>UPDATE</code> , <code>DELETE</code>	La fila que ya existe	La fila simplemente no aparece
<code>WITH CHECK</code>	<code>INSERT</code> , <code>UPDATE</code>	La fila como quedaría después	Error explícito, la escritura se rechaza

La asimetría importa y confunde:

```
-- Intento leer un centro de otra comuna:  
SELECT * FROM Centers WHERE center_id = 'VINA-C001';  
--> 0 filas. NO da error. Para ti, ese centro no existe.  
  
-- Intento crear un centro a nombre de otra comuna:  
INSERT INTO Centers (name, municipality_id) VALUES ('Trampa', 99);  
--> ERROR: new row violates row-level security policy for table "centers"
```

Leer de más devuelve vacío; escribir de más revienta. Por eso el backend traduce el código de error `42501` a un 403 HTTP, y los 0 resultados a un 404.

Un UPDATE pasa por las dos

Un `UPDATE` es leer y escribir a la vez, así que PostgreSQL aplica `USING` a la fila original y `WITH CHECK` a la fila resultante. Eso bloquea el ataque de "mover una fila a otra comuna":

```
-- USING dice: sí, ese centro es tuyo, puedes tocarlo.  
-- WITH CHECK dice: pero NO puedes dejarlo perteneciendo a la comuna 7.  
UPDATE Centers SET municipality_id = 7 WHERE center_id = 'VALPO-C003';  
--> ERROR: new row violates row-level security policy
```

Varias políticas sobre la misma tabla se suman con OR

Por defecto las políticas son **permisivas**: si hay tres políticas de `SELECT`, basta que **una** se cumpla para ver la fila. Se suman, no se restan.

`CenterItemPriority` (las necesidades de cada centro) tiene exactamente tres, y el resultado es la unión de las tres:

```

-- 1) Mis propios centros (db/002b)
CREATE POLICY cip_own_read ON CenterItemPriority FOR SELECT
  USING (center_id IN (SELECT c.center_id FROM Centers c
                        WHERE c.municipality_id = current_tenant()));

-- 2) El mapa público, sin sesión, solo centros activos (db/002b)
CREATE POLICY cip_public_read ON CenterItemPriority FOR SELECT
  USING (is_public_context()
        AND center_id IN (SELECT c.center_id FROM Centers c WHERE c.is_active = TRUE));

-- 3) Centros de otras comunas que comparten un SuperEvento conmigo (db/002d)
CREATE POLICY cip_intermunicipal_read ON CenterItemPriority FOR SELECT
  USING (center_id IN (SELECT s.center_id FROM super_event_shared_center_ids() s));

```

Que se sumen con OR tiene una consecuencia peligrosa que se explica más abajo (problema P2): si por error se activara el contexto público dentro de una sesión autenticada, la política 2 se sumaría a la 1 y un administrador vería los centros activos **de todas las comunas**. Por eso los contextos son mutuamente excluyentes.

El truco de "RLS activo y ninguna política" = denegar todo

Como sin políticas no se ve nada, encender RLS y no escribir ninguna política es la forma de **cerrar una tabla por completo**. Se usó para `RefreshTokens`:

```

-- Nadie, por ninguna consulta SQL de la aplicación, puede tocar esta tabla.
-- El único acceso es por funciones controladas (sección 8).
ALTER TABLE RefreshTokens ENABLE ROW LEVEL SECURITY;
ALTER TABLE RefreshTokens FORCE ROW LEVEL SECURITY;
-- (y aquí, deliberadamente, NO va ningún CREATE POLICY)

```

Ni un error de programación ni una inyección SQL pueden leer los tokens de sesión: no hay puerta que forzar.

4. Por qué hubo que crear `appcopio_app`

Esta es la parte que parece burocracia y en realidad es **la condición sin la cual nada de lo anterior existe**.

La regla de PostgreSQL que lo obliga

Un superusuario de PostgreSQL ignora RLS. Siempre. Aunque la tabla tenga `FORCE ROW LEVEL SECURITY`.

No hay forma de apagar eso. Es una decisión de diseño del motor: el superusuario existe justamente para poder rescatar la base cuando algo sale mal, así que ninguna política lo detiene.

El AppCopio single-tenant se conectaba como `postgres`, el superusuario que crea Docker por defecto. Si eso se hubiera dejado así, **las 38 tablas con RLS habrían quedado exactamente igual de desprotegidas que antes**, y peor: con la falsa sensación de estar protegidas. Todas las políticas escritas, ninguna aplicada.

Hay un segundo atributo que produce el mismo desastre: `BYPASSRLS`. Un rol con ese atributo tampoco pasa por las políticas.

El rol nuevo

```
-- db/002a_multitenant_schema.sql, sección 12
DO $$
BEGIN
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'appcopio_app') THEN
    CREATE ROLE appcopio_app LOGIN PASSWORD 'CAMBIA_ESTA_PASSWORD_EN_.env';
  END IF;
END $$;

-- Puede hacer su trabajo: leer y escribir datos.
GRANT USAGE ON SCHEMA public TO appcopio_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO appcopio_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO appcopio_app;

-- Y sobre las tablas que se creen DESPUÉS, para no tener que acordarse.
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO appcopio_app;
```

Lo clave es lo que **no** lleva: `CREATE ROLE` sin más no otorga `SUPERUSER` ni `BYPASSRLS`. El comentario en el archivo lo deja por escrito: "No cambiar esto: sin ello todo el RLS de arriba es decorativo."

Entonces, ¿quién usa cada rol?

Rol	Quién lo usa	Pasa por RLS	Para qué
<code>postgres</code> (superusuario)	Solo los scripts de <code>db_init/</code> al arrancar el contenedor	No	Crear tablas, roles y políticas; sembrar datos
<code>appcopio_app</code>	El backend, siempre	Sí	Toda la operación normal

Hace falta que sean dos porque **crear un rol requiere ser superusuario**: `appcopio_app` no puede crearse a sí mismo. El superusuario levanta el escenario; después se va y no vuelve.

En la práctica eso se ve así:

```
# docker-compose.yml - solo para inicializar la base
POSTGRES_USER: postgres
POSTGRES_DB: appcopio_mt_db
```

```
# backend/.env - lo que usa la aplicación
DB_USER=appcopio_app
DB_NAME=appcopio_mt_db
```

Y en el código, el valor por defecto también cambió, porque un defecto malo es una trampa esperando:

```
// backend/src/config/db.ts
// YA NO 'postgres' por defecto: los superusuarios ignoran RLS incluso con FORCE.
user: process.env.DB_USER || 'appcopio_app',
```

Cómo comprobarlo en 5 segundos

```
SELECT rolname, rolsuper, rolbypassrls
FROM pg_roles WHERE rolname = 'appcopio_app';
-- rolname      | rolsuper | rolbypassrls
-- -----+-----+-----
-- appcopio_app| f        | f           ← las dos en 'f' o el aislamiento es falso
```

Si alguna vez ves `t` en cualquiera de esas dos columnas, deja todo lo demás y arregla eso primero.

5. Cómo la base de datos sabe de qué comuna eres

Las políticas de la sección 3 dicen `municipality_id = current_tenant()`. Falta explicar de dónde sale ese `current_tenant()`, porque ahí está el engranaje central.

El detalle importante: **el backend se conecta siempre con el mismo usuario de base de datos** (`appcopio_app`), sea quien sea la persona que usó la aplicación. Así que la comuna no puede deducirse del usuario de PostgreSQL. Hay que **decírsela a la sesión**, en cada petición.

Variables de sesión: el recado que se le deja a la conexión

PostgreSQL permite guardar variables arbitrarias en la sesión y leerlas después. Son como un post-it pegado a la conexión.

```
-- Dejar el recado (el tercer parámetro true = LOCAL, dura hasta el fin de la transacción)
SELECT set_config('app.current_tenant', '3', true);

-- Leerlo (el segundo parámetro true = no reventar si no existe, devolver NULL)
SELECT current_setting('app.current_tenant', true); --> '3'
```

El prefijo `app.` es una convención: PostgreSQL acepta cualquier variable con un punto en el nombre, y `app.` deja claro que es de la aplicación y no del motor.

Las tres funciones de contexto

En vez de repetir `current_setting(...)` en cada política, se envuelve en tres funciones pequeñas:

```
-- db/002a_multitenant_schema.sql, sección 6

-- ¿De qué comuna es esta sesión? NULL si no se fijó ninguna.
-- NULLIF(..., '') convierte la cadena vacía en NULL antes de castear a INT,
-- porque '::INT daría error.
CREATE OR REPLACE FUNCTION current_tenant() RETURNS INT AS $$
    SELECT NULLIF(current_setting('app.current_tenant', true), '')::INT;
$$ LANGUAGE sql STABLE;

-- ¿Es el Super Administrador? (no tiene comuna; solo las políticas que lo
-- nombran expresamente lo admiten: no ve datos operativos de ninguna comuna)
CREATE OR REPLACE FUNCTION is_superadmin() RETURNS BOOLEAN AS $$
    SELECT COALESCE(current_setting('app.is_superadmin', true), 'false') = 'true';
$$ LANGUAGE sql STABLE;

-- ¿Es una visita anónima del mapa público?
CREATE OR REPLACE FUNCTION is_public_context() RETURNS BOOLEAN AS $$
    SELECT COALESCE(current_setting('app.public_access', true), 'false') = 'true';
$$ LANGUAGE sql STABLE;
```

STABLE le dice al planificador que dentro de una misma sentencia la función devuelve siempre lo mismo, así que puede evaluarla una vez en vez de una por fila. Sin eso, una tabla de 100.000 filas haría 100.000 llamadas.

Por qué **LOCAL** — y por qué obliga a usar transacciones

El **true** final de **set_config** significa **LOCAL**: el valor vive **solo hasta el COMMIT o ROLLBACK**. Y eso no es un detalle, es una decisión de seguridad.

Las conexiones de un *pool* se reciclan: la conexión que atendió a Valparaíso vuelve al pool y la toma la petición siguiente, que puede ser de Viña. Si el tenant fuera de sesión y no local, Viña heredaría el post-it de Valparaíso.

Consecuencia directa: para que **SET LOCAL** sirva de algo, **toda la petición tiene que correr dentro de una transacción explícita**. De ahí nace el middleware de la sección 6. Un **pool.query()** suelto fuera de esa transacción perdería el tenant en la sentencia siguiente.

La trampa de sintaxis que costó tiempo

El instinto es escribir esto, y **no compila**:

```
SET LOCAL app.current_tenant = $1;
-- ERROR: syntax error at or near "$1"
```

El comando **SET** de PostgreSQL solo acepta literales o identificadores, **no parámetros de bind**. Y concatenar el valor a mano (**SET LOCAL app.current_tenant = \${id}**) sería abrir la puerta a inyección SQL justo en el mecanismo que protege todo.

La forma parametrizable es la función:

```
// backend/src/auth/tenantContext.ts
function setLocal(client: PoolClient, name: string, value: string) {
  // set_config(nombre, valor, is_local) SÍ acepta parámetros $1/$2.
  return client.query('SELECT set_config($1, $2, true)', [name, value]);
}
```

6. Los middlewares: quién le cuenta a la base quién eres

Un **middleware** de Express es una función que se ejecuta *antes* del manejador de la ruta y puede enriquecer la petición, cortarla o dejarla pasar con `next()`. Se encadenan en orden.

En AppCopio multi-tenant la cadena típica es:

```
// backend/src/index.ts
app.use('/api/emergencias', requireAuth, withTenant, emergencyRoutes);
//                               ↑           ↑           ↑
//                               |           |           |   manejadores de la ruta
//                               |           |           |   abre transacción y fija el tenant
//                               |           |           |   verifica el JWT y rellena req.user
```

`requireAuth` — ¿quién eres?

Lee la cabecera `Authorization: Bearer <token>`, verifica la firma del JWT y deja el contenido en `req.user`. Si no hay token o está vencido, responde 401 y el request termina ahí.

El JWT lleva `user_id`, `role_id`, `es_apoyo_admin` y `municipality_id`. Que vaya firmado es lo que lo hace confiable: el cliente no puede cambiarlo sin invalidar la firma. Por eso existe la regla dura del proyecto:

La comuna **nunca** se toma del cuerpo de la petición. Siempre de `req.user.municipality_id`.

Hay una variante, `optionalAuth`, para rutas de doble uso (el listado de centros alimenta el mapa público y el panel municipal): si hay token válido rellena `req.user`, si no, sigue como anónimo en vez de cortar.

`withTenant` — contárselo a PostgreSQL

Es el middleware central de toda la migración. Hace cuatro cosas:

```
// backend/src/auth/tenantContext.ts (simplificado)
export async function withTenant(req, res, next) {
  const user = req.user;
  const isSuperadmin = user?.role_id === 4;

  // 1. Un usuario municipal sin comuna no puede resolverse a ningún tenant.
  // Antes caía a un tenant '-1' y el usuario veía TODO vacío sin entender por qué.
  if (!isSuperadmin && user?.municipality_id == null) {
    res.status(401).json({ error: 'TENANT_MISSING', message: 'Vuelve a iniciar sesión.' });
    return;
  }

  const client = await pool.connect();
  try {
    // 2. UNA transacción para toda la petición (obligatorio: SET LOCAL vive dentro de ella).
    await client.query('BEGIN');

    // 3. Fijar el contexto.
    if (isSuperadmin) await setLocal(client, 'app.is_superadmin', 'true');
    else await setLocal(client, 'app.current_tenant', String(user.municipality_id));

    // 4. Correr el resto del request con ESE cliente, y cerrar al terminar.
    runWithClient(client, res, next);
  } catch (err) {
    await abandonTx(client);
    next(err);
  }
}
```

El cierre es automático, colgado de los eventos de la respuesta:

```
async function finishTx(store, statusCode) {
  if (store.settled) return; // 'finish' y 'close' pueden dispararse ambos
  store.settled = true;

  const shouldRollback = statusCode >= 400 || store.rollbackRequested === true;
  await store.client.query(shouldRollback ? 'ROLLBACK' : 'COMMIT');
  store.client.release();
}
```

O sea: **si la respuesta sale con un código de error, la transacción se revierte entera**. Eso da atomicidad gratis a operaciones compuestas — por ejemplo, crear un SuperEvento e invitar comunas: si la invitación falla, no queda un SuperEvento huérfano.

Los tres modos de contexto

Middleware	Cuándo	Qué fija	Qué ve
<code>withTenant</code>	Rutas autenticadas	<code>app.current_tenant</code> o <code>app.is_superadmin</code>	Lo de su comuna (el Super Administrador, solo lo que administra: usuarios, SuperEventos, emergencias para agruparlas)
<code>withPublicContext</code>	Rutas públicas sin sesión	<code>app.public_access</code>	Solo lo que las políticas públicas permiten

Middleware	Cuándo	Qué fija	Qué ve
<code>withTenantOrPublic</code>	Rutas de doble uso	Uno u otro, nunca ambos	Según haya sesión o no

```
export async function withTenantOrPublic(req, res, next) {
  // Nunca los dos a la vez: si se activara public_access dentro de una request
  // autenticada, las policies permisivas se combinarían con OR y se filtrarían
  // datos de otras comunas.
  if (req.user) return withTenant(req, res, next);
  return withPublicContext(req, res, next);
}
```

Ese comentario describe un problema real que se encontró y se corrigió (P2 en la sección 10).

Los guardias de rol, que son otra cosa

No hay que confundir el contexto de tenant con los permisos por rol. Son capas distintas:

- `withTenant` responde "¿de qué comuna son los datos que puedes tocar?" → lo resuelve PostgreSQL.
- `requireSuperAdmin`, `requireTenant`, `soloAdminOApoyo` responden "¿tienes el cargo para hacer esta acción?" → lo resuelve el backend.

```
// backend/src/auth/requireUser.ts
// Devuelve la comuna del usuario. Es la ÚNICA fuente válida de municipality_id
// para cualquier escritura: nunca se toma del body.
export function requireTenant(req: Request): number {
  const u = requireUser(req);
  if (u.municipality_id == null) {
    const e = new Error('TENANT_REQUIRED');
    (e as any).status = 403;
    // El Super Administrador no tiene comuna, así que no puede crear datos operativos.
    throw e;
  }
  return u.municipality_id;
}
```

7. El interceptor del pool: cómo no reescribir 31 servicios

Hasta aquí hay un problema práctico grande. `withTenant` fija el tenant en **un cliente concreto** del pool. Pero el código del repo está lleno de esto:

```
import pool from '../config/db';
// ...
const { rows } = await pool.query('SELECT * FROM Centers');
//           ↑ ¡esto pide una conexión CUALQUIERA del pool!
```

Esa conexión cualquiera **no tiene el tenant fijado**, porque el `SET LOCAL` se hizo en otra. Resultado: cero filas, o peor, comportamiento errático según qué conexión toque.

El repo tenía, al momento de migrar:

- **31 llamadas a** `pool.connect()` repartidas en 11 routers,
- **13 archivos con sus propios** `BEGIN` / `COMMIT` / `ROLLBACK`,
- **23 servicios con** `db: Db` **inyectado y 8 que importan** `pool` **directo** — dos estilos conviviendo.

Reescribirlos todos a mano habría sido semanas de trabajo y una fuente enorme de errores por omisión.

La solución: `AsyncLocalStorage` + un pool que miente amablemente

`AsyncLocalStorage` es una utilidad de Node que mantiene un "contexto" disponible durante toda una cadena de llamadas asíncronas, sin pasarlo por parámetro. Es, en esencia, una variable global **por petición**.

```
// backend/src/config/db.ts
export type TenantStore = { client: PoolClient; rollbackRequested?: boolean };
export const tenantStorage = new AsyncLocalStorage<TenantStore>();
```

`withTenant` guarda ahí el cliente con el tenant fijado. Y entonces se **intercepta** `pool.query` una sola vez, en el módulo de configuración:

```
const originalQuery = pool.query.bind(pool);

(pool as any).query = (textOrConfig, params?, callback?) => {
  const store = tenantStorage.getStore();

  if (store) {
    // Estamos dentro de un request que ya pasó por withTenant.

    // Los BEGIN/COMMIT/ROLLBACK internos de los servicios viejos se vuelven no-op:
    // la transacción real la controla el middleware.
    if (isControlStatement(textOrConfig)) {
      const t = textOrConfig.trim().toUpperCase();
      // Un ROLLBACK interno no puede revertir por sí solo, así que se ANOTA
      // para que finishTx cierre con ROLLBACK al final del request.
      if (t === 'ROLLBACK') store.rollbackRequested = true;
      const fake = fakeControlResult(t);
      return callback ? callback(null, fake) : Promise.resolve(fake);
    }

    // Cualquier otra consulta va al cliente CON el tenant fijado.
    return store.client.query(textOrConfig, params, callback);
  }

  // Fuera de un request (scripts, arranque): comportamiento normal.
  return originalQuery(textOrConfig, params, callback);
};
```

Y también `pool.connect()`, que devuelve un **Proxy** sobre el mismo cliente en vez de abrir una conexión nueva:

```
(pool as any).connect = async (...args) => {
  const store = tenantStorage.getStore();
  if (store) {
    return new Proxy(store.client, {
      get(target, prop, receiver) {
        // .release() se vuelve no-op: lo libera el middleware al final del request.
        // Si un servicio liberara la conexión a mitad, el tenant se perdería.
        if (prop === 'release') return () => {};
        if (prop === 'query') { /* mismo filtro de BEGIN/COMMIT/ROLLBACK */ }
        return Reflect.get(target, prop, receiver);
      },
    });
  }
  return originalConnect(...args);
};
```

Qué se ganó con esto

Ni un solo servicio tuvo que cambiar. El código que escribe `pool.query(...)` o `pool.connect()` sigue viendo lo mismo, pero ahora corre dentro de la transacción del request, con el tenant puesto y sujeto a RLS. Los dos estilos que conviven en el repo quedan cubiertos por el mismo mecanismo.

El detalle del `rollbackRequested`

Merece una línea aparte porque es sutil. Si un servicio hace su propio `ROLLBACK`, ese comando se anula (es no-op) — pero el servicio lo pidió por una razón. Sin la marca, el middleware haría `COMMIT` al final y **se guardaría justo lo que el servicio quiso descartar**. La bandera propaga la intención hasta el cierre real.

8. `SECURITY DEFINER`: la válvula de escape controlada

RLS aísla tan bien que aparece un problema nuevo: **a veces hay que leer legítimamente más allá del propio tenant**, y desde dentro de una política eso es imposible — cualquier subconsulta también pasa por RLS y devuelve solo lo propio.

Qué significa

Toda función de PostgreSQL corre en uno de dos modos:

Modo	Con qué permisos corre	Es el defecto
<code>SECURITY INVOKER</code>	Los de quien la llama — pasa por RLS normalmente	Sí
<code>SECURITY DEFINER</code>	Los de quien la creó (aquí, el dueño de las tablas) — no pasa por RLS	No

La analogía: `SECURITY DEFINER` es una ventanilla. Tú no entras al archivo; le pides algo concreto a la persona que sí puede entrar, y ella decide si te lo entrega. **La función es la ventanilla, y su cuerpo es la política de la ventanilla.**

Por eso el peligro es real: dentro de una función `SECURITY DEFINER` **el aislamiento no existe**. Lo que la vuelve segura es que sea **estrecha**: pocos parámetros, sin texto libre, con verificación explícita adentro y devolviendo solo las columnas justas.

Todas llevan además `SET search_path = public`, para que nadie pueda anteponer un esquema propio con tablas falsas y secuestrar la función.

Caso 1 — El login, que no puede tener tenant todavía

El huevo y la gallina: para saber de qué comuna es alguien hay que leer `Users`; pero `Users` tiene RLS por comuna; pero la comuna aún no se conoce. Con RLS activo, un `SELECT` directo devuelve 0 filas y **el login falla para todo el mundo**.

```
-- db/002a, sección 6b. La ÚNICA vía de lectura de Users sin tenant,
-- y está acotada a un username exacto: no permite listar ni explorar.
CREATE OR REPLACE FUNCTION auth_lookup_user(p_username TEXT)
RETURNS TABLE (user_id INT, username TEXT, password_hash TEXT, is_active BOOLEAN,
                role_id INT, es_apoyo_admin BOOLEAN, /* ... */ municipality_id INT,
                municipality_shortcode TEXT, municipality_name TEXT, role_name TEXT)
LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE AS $$
  SELECT u.user_id, u.username::TEXT, /* ... */ m.name::TEXT, r.role_name::TEXT
  FROM Users u
  LEFT JOIN Roles r ON r.role_id = u.role_id
  LEFT JOIN Municipalities m ON m.municipality_id = u.municipality_id
  WHERE u.username = p_username; -- ← exacto, no LIKE, no sin WHERE
$$;
```

Caso 2 — Encapsular una tabla cerrada a cal y canto

Recuerda `RefreshTokens`: RLS activo, cero políticas, nadie entra. Sus siete operaciones se convirtieron en siete funciones:

```
-- db/002c, sección 5
CREATE OR REPLACE FUNCTION refresh_token_find(p_user_id INT, p_token_hash TEXT)
RETURNS TABLE (id BIGINT, user_id INT, expires_at TIMESTAMPTZ, revoked_at TIMESTAMPTZ)
LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE AS $$
  SELECT rt.id, rt.user_id, rt.expires_at, rt.revoked_at
  FROM RefreshTokens rt
  WHERE rt.user_id = p_user_id AND rt.token_hash = p_token_hash;
$$;
-- Fijate que NO devuelve token_hash: ni siquiera la ventanilla entrega el secreto.
```

Caso 3 — Un trigger que necesita escribir donde el usuario no puede

Los identificadores de centro son `VALPO-C001`, con correlativo por comuna. El trigger que los genera incrementa un contador en `Municipalities`, tabla que solo el Super Administrador puede modificar:

```

-- db/002a, sección 5
-- SECURITY DEFINER es obligatorio: el trigger hace UPDATE sobre Municipalities,
-- que tiene FORCE RLS. Sin esto, el UPDATE afecta 0 filas, v_seq queda NULL
-- y ningún admin municipal puede crear un centro.
CREATE OR REPLACE FUNCTION generate_center_id() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE v_shortcode TEXT; v_seq INT;
BEGIN
    IF NEW.center_id IS NOT NULL THEN RETURN NEW; END IF;

    UPDATE Municipalities
        SET center_seq_counter = center_seq_counter + 1
        WHERE municipality_id = NEW.municipality_id
        RETURNING center_seq_counter, shortcode INTO v_seq, v_shortcode;

    IF v_seq IS NULL THEN
        RAISE EXCEPTION 'Municipality % no existe', NEW.municipality_id;
    END IF;

    NEW.center_id := v_shortcode || '-C' || LPAD(v_seq::text, 3, '0');
    RETURN NEW;
END;
$$;

```

Caso 4 — La colaboración entre comunas

Aquí está el uso más interesante, y el que muestra cómo se hace una ventanilla **segura**. El tablero intercomunal tiene que mostrar centros de otras comunas — pero solo algunos, y solo algunas columnas.

```

-- db/002d, 7c. Devuelve SOLO lo que la regla del proyecto permite cruzar:
-- ubicación, capacidad, % de llenado y estado operacional.
-- NUNCA CentersDescription, FamilyGroups, Persons ni cantidades de inventario.
CREATE OR REPLACE FUNCTION super_event_shared_centers(p_super_event_id INT)
RETURNS TABLE (
  center_id VARCHAR, name TEXT, latitude DECIMAL, longitude DECIMAL,
  capacity INT, fullness_percentage INT, operational_status TEXT,
  municipality_id INT, municipality_shortcode TEXT, activation_id INT,
  emergency_id INT, emergency_name TEXT
) -- ↑ el límite está escrito AQUÍ, en la base, no en TypeScript
LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE AS $$
  SELECT c.center_id, c.name::TEXT, c.latitude, c.longitude, /* ... */
  FROM Centers c
  JOIN CentersActivations ca ON ca.center_id = c.center_id AND ca.ended_at IS NULL
  JOIN Emergencies e ON e.emergency_id = ca.emergency_id
    AND e.super_event_id = p_super_event_id
  JOIN SuperEvents se ON se.super_event_id = e.super_event_id
    AND se.ended_at IS NULL -- el evento sigue abierto
  JOIN SuperEventParticipants duena -- la comuna DUEÑA participa
    ON duena.super_event_id = se.super_event_id
    AND duena.municipality_id = c.municipality_id
    AND duena.status = 'participando'
  WHERE c.is_active = TRUE
    AND EXISTS ( -- y QUIEN LEE también participa
      SELECT 1 FROM SuperEventParticipants yo -- (sin excepción para el superadmin)
      WHERE yo.super_event_id = p_super_event_id
        AND yo.municipality_id = current_tenant()
        AND yo.status = 'participando'
    );
$$;

```

Los cuatro candados de esa función: el evento abierto, la comuna dueña participando, la comuna lectora participando, y el conjunto de columnas fijado en la firma. **Que el límite de columnas viva en la base y no en el servicio significa que nadie puede ampliarlo por descuido escribiendo TypeScript.**

Hay una variante del mismo patrón para **escribir** en otra comuna: `notify_super_event_invitation` y `notify_support_offer` existen porque la política de notificaciones solo deja escribir avisos para la propia comuna, y una invitación tiene que aterrizar en la de enfrente. Ambas verifican antes de escribir:

```

-- db/002d, 7e. Lo único que impide notificar a nombre de otra comuna,
-- porque la lectura de arriba omitió RLS por ser SECURITY DEFINER.
IF v_offer.from_municipality_id IS DISTINCT FROM current_tenant() THEN
  RAISE EXCEPTION 'SOLO_LA_COMUNA_DE_ORIGEN_PUEDE_AVISAR';
END IF;

```

Los permisos de las funciones

Crear la ventanilla no basta; hay que decidir quién puede tocar el timbre. Por defecto **cualquiera** puede ejecutar una función nueva, así que se revoca y se otorga explícitamente:

```

REVOKE ALL ON FUNCTION super_event_shared_centers(INT) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION super_event_shared_centers(INT) TO appcopio_app;

```

9. Cómo se cubrieron 38 tablas: cuatro patrones

No todas las tablas se parecen. `Centers` tiene comuna propia; `DatasetFieldOptions` cuelga de un campo, que cuelga de un dataset, que sí tiene comuna. Se usó el patrón más barato en cada caso.

Patrón A — Columna directa (lo más simple)

La tabla lleva su propio `municipality_id`. Como son varias tablas idénticas, se aplicó en bucle:

```
-- db/002a, sección 7
-- Ojo: los nombres van en minúscula. Las tablas se crearon sin comillas, así que
-- en pg_class son 'users', 'centers', etc. format('%I','Users') produciría "Users"
-- (con comillas) y fallaría: esa tabla no existe.
DO $$
DECLARE t TEXT;
BEGIN
  FOREACH t IN ARRAY ARRAY['users','centers','centersactivations','familygroups',
                           'persons','centerinventoryitems','datasets'] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
    EXECUTE format(
      'CREATE POLICY %I_tenant_isolation ON %I
       USING      (municipality_id = current_tenant())
       WITH CHECK (municipality_id = current_tenant())',
      t, t);
  END LOOP;
END $$;
```

Caso especial: `Persons` lleva columna propia aunque podría deducirla por subconsulta. Es dato sensible que nunca cruza de comuna, así que se prefiere un `WITH CHECK` real (no `true`) y evitar un join de tres tablas por fila en cada lectura.

Patrón B — Catálogo dual: `NULL` significa "de todos"

Productos, categorías, plantillas y zonas pueden ser globales (sembrados por la plataforma) o propios de una comuna:

```
-- db/002a, sección 8
CREATE POLICY categories_tenant_or_global ON Categories
  USING      (municipality_id IS NULL          -- ← catálogo global, visible para todos
              OR municipality_id = current_tenant())
  WITH CHECK (municipality_id IS NULL
              OR municipality_id = current_tenant());
```

Eso obligó además a repensar los índices únicos. El original era `UNIQUE(name)` global, lo que impedía que dos comunas tuvieran un producto con el mismo nombre:

```
ALTER TABLE Products DROP CONSTRAINT products_name_key;

-- Únicos entre los globales...
CREATE UNIQUE INDEX products_name_global_uq ON Products (name)
  WHERE municipality_id IS NULL;
-- ...y únicos por comuna entre sí.
CREATE UNIQUE INDEX products_name_tenant_uq ON Products (name, municipality_id)
  WHERE municipality_id IS NOT NULL;
```

Patrón C — Subconsulta al padre (tablas satélite)

Para tablas de poco volumen que cuelgan de otra, no vale la pena agregar columna: la política pregunta al padre.

```
-- db/002c, sección 3
CREATE POLICY datasetfields_via_padre ON DatasetFields
  USING (dataset_id IN (SELECT d.dataset_id FROM Datasets d));
```

Lo elegante: esa subconsulta **también pasa por RLS**. `Datasets` ya está aislada, así que solo devuelve los datasets de mi comuna, y la política hereda el aislamiento sin repetir la condición. El aislamiento se propaga solo.

Patrón D — Columna + trigger de herencia (tablas hoja de volumen)

Para tablas grandes (`InventoryLog`, `DatasetRecords`, turnos, asignaciones), la subconsulta en cada lectura sería cara. Llevan columna propia, pero **rellenada automáticamente** para que ningún `INSERT` del backend tenga que cambiar:

```
-- db/002c, sección 1
-- Corre como INVOCADOR a propósito (NO SECURITY DEFINER): la consulta al padre
-- pasa por RLS, así que si el centro no es de tu comuna la subconsulta devuelve
-- NULL y el NOT NULL rechaza la fila. El aislamiento de ESCRITURA se aplica solo.
CREATE OR REPLACE FUNCTION heredar_comuna_de_centro() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.municipality_id IS NULL THEN -- solo si no vino explícito
    NEW.municipality_id := (
      SELECT c.municipality_id FROM Centers c WHERE c.center_id = NEW.center_id
    );
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_inventorylog_comuna BEFORE INSERT ON InventoryLog
  FOR EACH ROW EXECUTE FUNCTION heredar_comuna_de_centro();
```

Ese comentario esconde algo bonito: **el trigger se convierte en un mecanismo de seguridad por accidente feliz**. Si intentas registrar un movimiento de inventario sobre un centro ajeno, el `SELECT` interno no lo encuentra (RLS), la columna queda `NULL`, y el `NOT NULL` rechaza la fila.

Y el orden de las sentencias importa:


```
-- El NOT NULL se aplica DESPUÉS de crear los triggers: así el seed puede insertar
-- sin declarar la columna y el trigger la completa.
ALTER TABLE InventoryLog ALTER COLUMN municipality_id SET NOT NULL;
```

Resumen de dónde está cada cosa

Archivo	Qué contiene
db/001_tablas.sql	El esquema single-tenant original, sin tocar
db/002a_multitenant_schema.sql	Municipalities, rol appcopio_app, funciones de contexto, RLS del núcleo
db/002b_emergencias_y_notificaciones.sql	Notificaciones municipales, un solo admin activo, lecturas de prioridades
db/002c_qls_tablas_restantes.sql	Triggers de herencia, tablas satélite, RefreshTokens, ofertas
db/002d_supereventos.sql	SuperEventos y toda la colaboración intercomunal
db/003_datos.sql y 005_datos_validacion.sql	Datos sembrados

El orden alfabético **es** el orden de ejecución: Docker corre los scripts de ese directorio en ese orden. Por eso los sufijos **a**, **b**, **c**, **d**. Y por eso nunca se hace un **ALTER** a mano contra una base viva: el entorno se recrea entero con `docker compose down -v && docker compose up`.

10. Los tropezos reales (y qué enseña cada uno)

El catálogo completo está en `docs/02_implementacion_tecnica.md`, sección 6, con 40 problemas numerados. Aquí van los que mejor explican **cómo se comporta RLS en la práctica**, que es distinto de cómo se lee en la documentación.

P1 — Activar RLS sobre `Users` rompió el login de todos

Síntoma: nadie podía entrar. **Causa:** el login lee `Users` para averiguar la comuna, pero RLS ya exigía saber la comuna para leer `Users`. **Lección:** toda tabla que se consulta *antes* de conocer el tenant necesita una vía `SECURITY DEFINER` acotada.

P2 — Fuga de tenant por montar `withPublicContext` a nivel de prefijo

Síntoma: un administrador municipal veía centros activos de otras comunas. **Causa:** el middleware público se había montado sobre todo el prefijo `/api/centers`, así que `app.public_access` quedaba activo **también** en peticiones autenticadas — y como las políticas permisivas se suman con OR, la política pública ampliaba lo que veía el admin.

Lección: los contextos tienen que ser mutuamente excluyentes, y los routers que mezclan endpoints públicos y privados aplican el middleware **por ruta**, nunca por prefijo. En `index.ts` está escrito:

```
// Los routers que MEZCLAN endpoints públicos y privados bajo el mismo prefijo
// (centerRoutes, prioritiesRoutes, volunteerRoutes, serviceRequestRoutes) NO reciben
// middleware acá: lo aplican por ruta en su propio archivo.
app.use("/api/centers", centerRoutes);
```

P11 — Una comuna no podía leer sus propias prioridades

Causa: en el primer diseño la única política de `SELECT` sobre `CenterItemPriority` era la intercomunal. Al ser RLS "denegar por defecto", no tener política propia significaba **no ver lo propio**.

Lección: con RLS hay que acordarse de permitir explícitamente lo obvio. Lo que no está escrito, no existe.

P12 — El conteo de participantes siempre daba 1

Causa: `SELECT COUNT(*) FROM SuperEventParticipants WHERE super_event_id = 5` devolvía 1 porque la política `sep_read` solo deja ver **la fila propia**. El conteo era correcto: contaba lo que la comuna podía ver.

Lección: con RLS, los agregados (`COUNT`, `SUM`, `AVG`) se calculan **sobre las filas visibles**. Si necesitas contar lo que no puedes ver, necesitas una ventanilla. Se resolvió así:

```
(SELECT COUNT(*) FROM super_event_participants_of(se.super_event_id) p
 WHERE p.status = 'participando')::int AS total_participando
```

P19 — Fuga de conexiones que agotaba el pool y caía el proceso

Causa: la marca de "esta transacción ya se cerró" se guardaba en el objeto `PoolClient`. Pero **las conexiones se reciclan**: al liberarlas vuelven al pool y otra petición recibe el mismo objeto, ya marcado. Esa petición salía antes de tiempo y dejaba su transacción abierta para siempre.

Lección: nunca guardes estado por petición en un objeto que el pool reutiliza. El estado se movió al store del `AsyncLocalStorage`, que sí es por petición.

Del mismo problema salió otra pieza: hay que escuchar el evento `error` del cliente, porque si PostgreSQL corta la conexión y nadie escucha, Node lo trata como excepción no capturada y **se cae el proceso entero**.

P38 y P39 — Las dos trampas de RLS al invitar comunas

Estas dos son las más instructivas de todas, porque muestran que **RLS no solo filtra lecturas: cambia la forma del código que escribes**.

Al invitar una comuna se crea una fila cuyo `municipality_id` es el de la comuna **invitada**. La política `sep_read` solo deja ver la fila propia → **quien invita no puede ver la fila que acaba de crear**. De ahí salen dos consecuencias:

P38 — No se puede avisarle a la comuna invitada. La política de notificaciones solo deja escribir avisos para la propia comuna. Se resolvió con `notify_super_event_invitation`, una ventanilla que compone el mensaje en la base y exige que la fila de invitación ya exista.

P39 — `ON CONFLICT` bloquea un `INSERT` que el `WITH CHECK` sí permite. PostgreSQL exige que la fila nueva sea visible bajo la política de `SELECT` cuando hay `ON CONFLICT`, y aborta con *"new row violates row-level security policy"* aunque el `WITH CHECK` se cumpla. Se verificó que falla incluso con `WITH CHECK (true)`, y que deja de fallar si `sep_read` fuera `USING(true)`.

La solución final tiene tres piezas, y ninguna es obvia:

```
// backend/src/services/superEventService.ts
for (const raw of municipalityIds) {
  const municipalityId = Number(raw);

  // 1. SAVEPOINT: todo el request es UNA transacción, así que sin esto la
  // primera comuna repetida abortaría la transacción entera.
  await db.query('SAVEPOINT invitacion');
  try {
    // 2. INSERT a secas: sin ON CONFLICT (P39) y sin RETURNING (no se ve la fila).
    await db.query(
      `INSERT INTO SuperEventParticipants
      (super_event_id, municipality_id, status, invited_by, invited_by_municipality_id)
      VALUES ($1, $2, 'invitada', $3, $4)`,
      [superEventId, municipalityId, invitedBy, invitedByMunicipalityId]
    );
    await db.query('RELEASE SAVEPOINT invitacion');
    invitadas.push(municipalityId);
  } catch (err: any) {
    await db.query('ROLLBACK TO SAVEPOINT invitacion');
    await db.query('RELEASE SAVEPOINT invitacion');
    // 3. El ÚNICO modo fiable de saber que ya estaba invitada: las comprobaciones
    // de unicidad de Postgres. Sí ignoran RLS, así que el 23505 sí llega.
    if (err?.code === '23505') { yaEstaban.push(municipalityId); continue; }
    throw err;
  }
}
```

Un `SELECT` previo tampoco habría servido: no vería la fila ajena y siempre diría "no existe".

El patrón común de todos estos errores

Casi todos comparten una misma forma: **algo que funcionaba se volvió invisible**, y el síntoma fue cero filas en vez de un error. Con RLS activo, la primera hipótesis ante un resultado vacío no debe ser "el `WHERE` está mal" sino "¿qué política me lo está escondiendo, y estoy en el contexto correcto?".

11. Cómo comprobar que el aislamiento es real

La trampa de RLS es que **una implementación rota se ve exactamente igual que una correcta** mientras uses la aplicación con un solo usuario. Hay que ir a buscarla.

Paso 0 — Recrear el entorno desde cero

```
docker compose down -v && docker compose up
```

El `-v` borra los volúmenes. Sin él, los scripts de `db_init/` **no vuelven a correr** (Docker solo los ejecuta cuando el volumen está vacío) y estarías probando el esquema viejo.

Paso 1 — El rol no debe poder saltarse RLS

Si esto falla, todo lo demás da igual.

```
SELECT rolname, rolsuper, rolbypassrls FROM pg_roles WHERE rolname = 'appcopio_app';  
-- se espera: f | f
```

Paso 2 — Todas las tablas con RLS activo y forzado

```
-- Tablas SIN row security: deberían ser solo las que no llevan datos de comuna.  
SELECT relname, relrowsecurity, relforcerowsecurity  
FROM pg_class  
WHERE relkind = 'r' AND relnamespace = 'public'::regnamespace  
AND (relrowsecurity = false OR relforcerowsecurity = false)  
ORDER BY relname;
```

Busca especialmente `relforcerowsecurity = false` con `relrowsecurity = true`: esa combinación es la que deja al dueño pasar de largo.

Paso 3 — La prueba más directa: sin tenant, sin datos

Esta es la más contundente porque no depende de la aplicación en absoluto.

```
# Como appcopio_app, SIN fijar ningún tenant.  
docker compose exec -T db psql -U appcopio_app -d appcopio_mt_db -tAc \  
"SELECT COUNT(*) FROM Centers;"  
# se espera: 0
```

Si devuelve el total de centros, RLS no está haciendo nada. **Cero filas fuera de contexto es la señal de que el aislamiento no depende de que el backend recuerde filtrar.**

Paso 4 — Con tenant, solo lo propio

Recuerda que `SET LOCAL` vive dentro de la transacción, así que ambas sentencias deben ir en la misma:

```
docker compose exec -T db psql -U appcopio_app -d appcopio_mt_db -tAq \  
-c "BEGIN" \  
-c "SELECT set_config('app.current_tenant', '1', true)" \  
-c "SELECT COUNT(*) FROM Centers" \  
-c "COMMIT"  
# se espera: solo los centros de la comuna 1
```

Cambia el `1` por otra comuna y comprueba que el número cambia. Si da lo mismo en las dos, algo está mal.

Paso 5 — Que las escrituras cruzadas exploten

```
BEGIN;
SELECT set_config('app.current_tenant','1',true);
-- Intento crear un centro a nombre de la comuna 2:
INSERT INTO Centers (name, municipality_id, address, capacity)
VALUES ('Trampa', 2, 'x', 10);
-- se espera: ERROR ... violates row-level security policy
ROLLBACK;
```

El `ROLLBACK` es lo que hace segura la comprobación: aunque la política fallara y la fila entrara, no queda nada escrito.

Paso 6 — Por la API, que es lo que ve el usuario

Haz login como administrador de la comuna A y pide un recurso de la comuna B por su identificador. Lo esperado es **403, 404 o cero filas — nunca datos**. Repítelo con:

- un centro de otra comuna,
- una emergencia de otra comuna,
- el tablero de un SuperEvento en el que tu comuna está solo `invitada` (debe dar 403 `NO_PARTICIPA`),
- el mapa público sin sesión (debe seguir funcionando y no mostrar inventario ni personas).

El script que hace todo esto solo

```
docker compose down -v && docker compose up -d
bash scripts/validar_multitenant.sh
```

Cubre roles y RLS en la base, aislamiento por API, permisos por rol, gobernanza de administración, colaboración intercomunal, canal público e higiene de sesión. Ninguna comprobación deja datos detrás: las que ejercitan escrituras corren dentro de `BEGIN ... ROLLBACK`, y la única que escribe de verdad (crear una municipalidad, que solo se puede acreditar creándola) borra lo que creó al terminar.

Exige la base **recién recreada**, porque varias comprobaciones son conteos absolutos sobre los datos sembrados. Lo visual —mapa, badges, modales— va aparte, en `docs/03_guion_de_validacion.md`.

12. Chuleta

Glosario

Término	Qué es
Tenant	Cada organización que usa el sistema. Aquí, una municipalidad
Discriminador	La columna que dice de quién es cada fila: <code>municipality_id</code>
RLS	Row-Level Security: PostgreSQL filtra filas según una regla, automáticamente

Término	Qué es
Política	La regla concreta (<code>CREATE POLICY</code>). Varias se suman con OR
<code>USING</code>	Qué filas existentes puedes ver o tocar. Si falla: fila invisible
<code>WITH CHECK</code>	Cómo puede quedar una fila que escribes. Si falla: error
<code>ENABLE</code>	Enciende RLS para todos menos el dueño de la tabla
<code>FORCE</code>	Lo enciende también para el dueño. Sin esto, las políticas son decorativas
<code>SECURITY DEFINER</code>	Función que corre con permisos de quien la creó: no pasa por RLS
<code>SET LOCAL</code>	Variable de sesión que vive solo hasta el fin de la transacción
Middleware	Función de Express que corre antes del manejador de la ruta
<code>AsyncLocalStorage</code>	Contexto de Node disponible en toda una cadena asíncrona, por petición

Errores de PostgreSQL que verás

Código	Significa	En AppCopio se traduce a
<code>42501</code>	Una política bloqueó la operación	403
<code>23505</code>	Clave duplicada	409, o "ya estaba invitada"
<code>23503</code>	El padre no existe o no es visible	404
0 filas	Casi siempre RLS, no el <code>WHERE</code>	404

El recorrido completo de una petición

```

Navegador (Bearer token)
↓
requireAuth ..... verifica la firma del JWT → req.user
↓
withTenant ..... BEGIN + set_config('app.current_tenant', ...)
↓
Guardia de rol ..... requireTenant / soloAdminOApoyo
↓
Servicio ..... SQL escrito SIN filtro de comuna
↓
pool interceptado ... redirige al cliente con el tenant fijado
↓
PostgreSQL ..... aplica RLS y devuelve solo lo permitido
↓
finishTx ..... COMMIT si status < 400, ROLLBACK si no

```

Las diez reglas de oro

1. **La comuna sale del JWT, nunca del body.** Siempre `req.user.municipality_id`.
2. **`FORCE` en toda tabla con `municipality_id`.** Sin él, el dueño se salta las políticas.

3. **El backend nunca se conecta como superusuario.** Si `rolsuper` o `rolbypassrls` dan `t`, el aislamiento es ficticio.
4. **Sin transacción no hay tenant.** `SET LOCAL` muere en el `COMMIT`; un `pool.query()` fuera de la transacción del middleware pierde el contexto.
5. **Los contextos son excluyentes.** Tenant o público, jamás los dos: las políticas permisivas se suman con OR.
6. **`SECURITY DEFINER` solo para ventanillas estrechas.** Pocos parámetros, sin texto libre, con verificación dentro y columnas fijadas en la firma.
7. **Ante 0 filas, sospecha de la política antes que del `WHERE`.**
8. **Los agregados cuentan solo lo visible.** Un `COUNT` bajo RLS no es el total real.
9. **Cada cambio de esquema es un `.sql` nuevo,** nunca un `ALTER` a mano contra una base viva. El entorno se recrea con `docker compose down -v`.
10. **Los datos sensibles de personas jamás cruzan de comuna,** ni en colaboración. Entre comunas solo viajan ubicación, capacidad, llenado, estado operacional y prioridades.

Dónde seguir leyendo

Documento	Para qué
<code>docs/01_modelo_de_negocio.md</code>	Las decisiones de dominio: qué se comparte y por qué
<code>docs/02_implementacion_tecnica.md</code>	El detalle técnico y los 40 problemas numerados
<code>docs/03_guión_de_validacion.md</code>	El guión de validación visual, pantalla por pantalla
<code>docs/04_entidades_y_flujo_multitenant.md</code>	Entidades y flujos de emergencias y SuperEventos
<code>AppCopio_MultiTenant_Plan.md</code>	El plan de migración paso a paso
<code>CLAUDE.md</code>	Las reglas duras del proyecto, en corto